

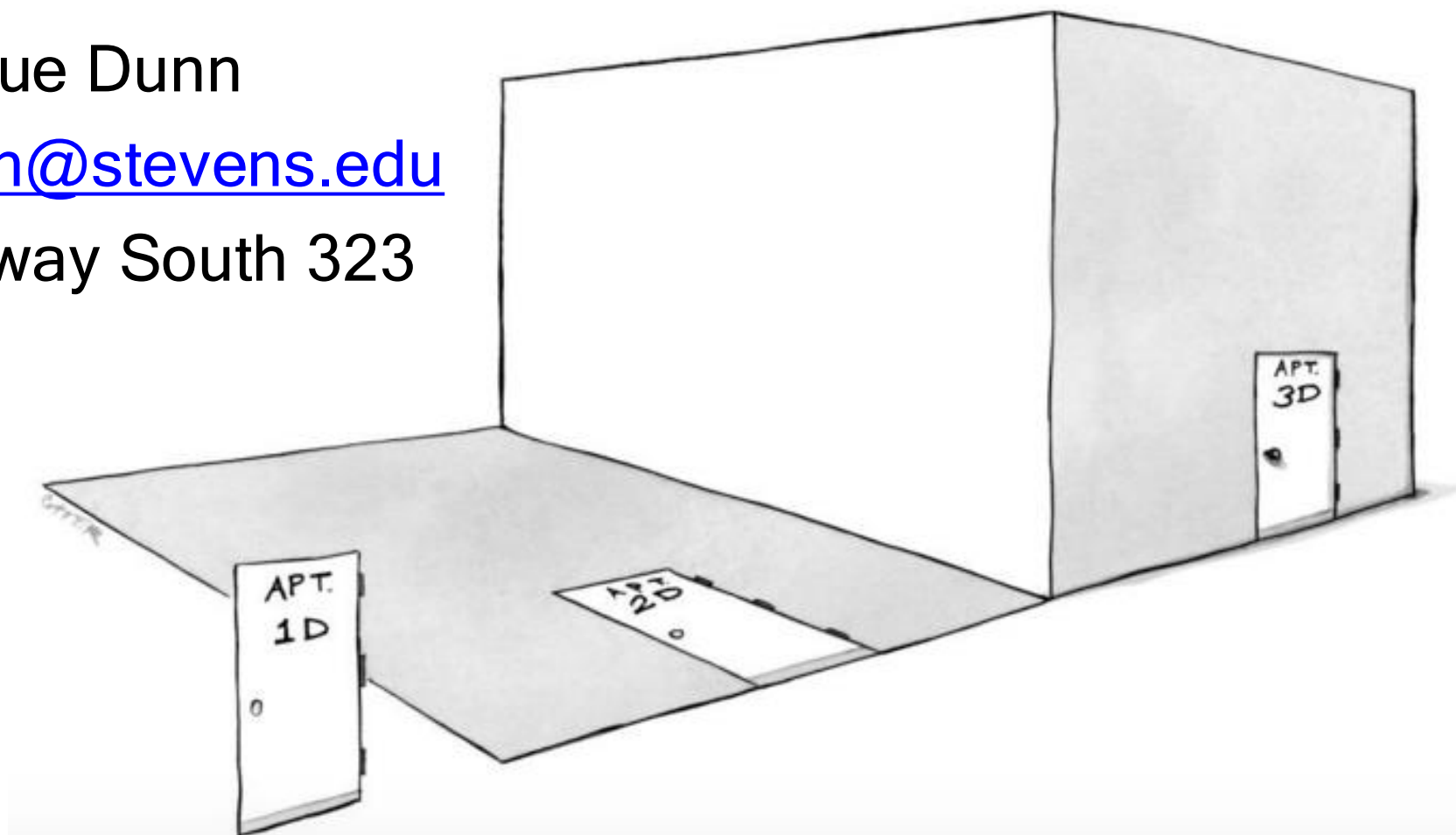
CS 532: 3D Computer Vision

Lecture 2

Enrique Dunn

edunn@stevens.edu

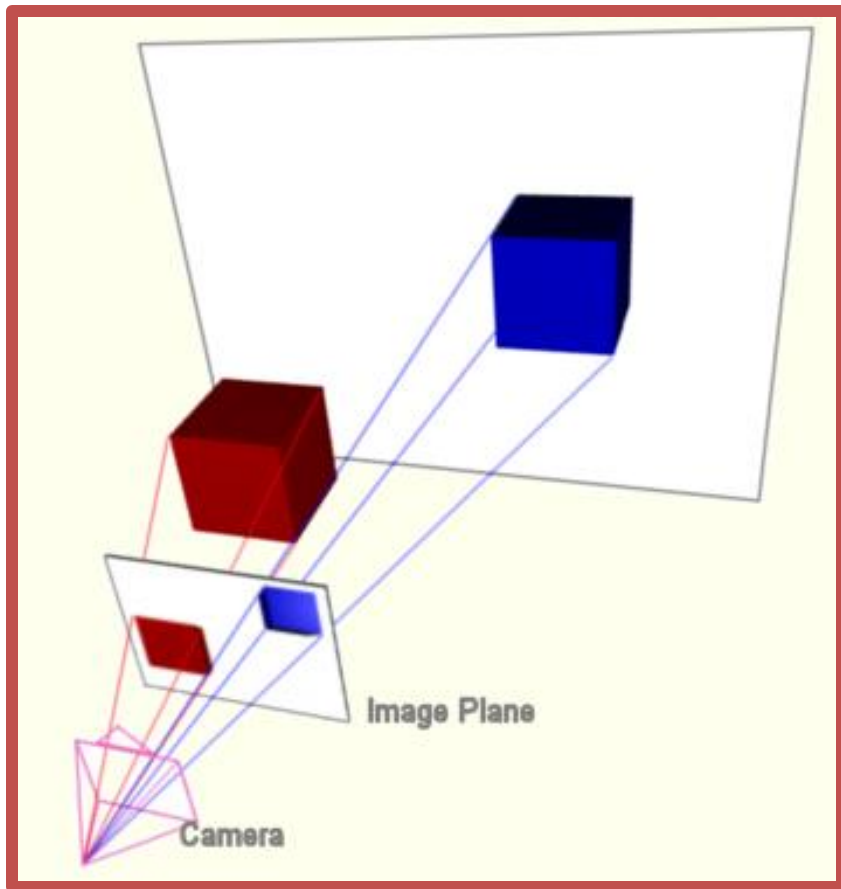
Gateway South 323



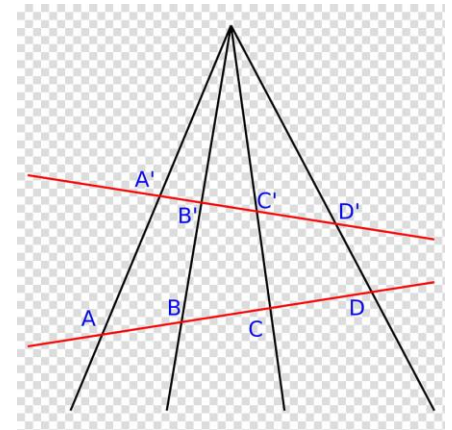
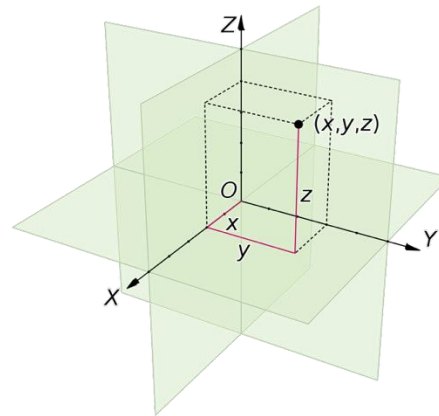
Introduction to Geometry

Based on slides by M. Pollefeys (ETH)
and D. Capperli (Purdue)

Pin-hole Camera Model



- Euclidean Geometry
- Projective Geometry
- Linear Algebra

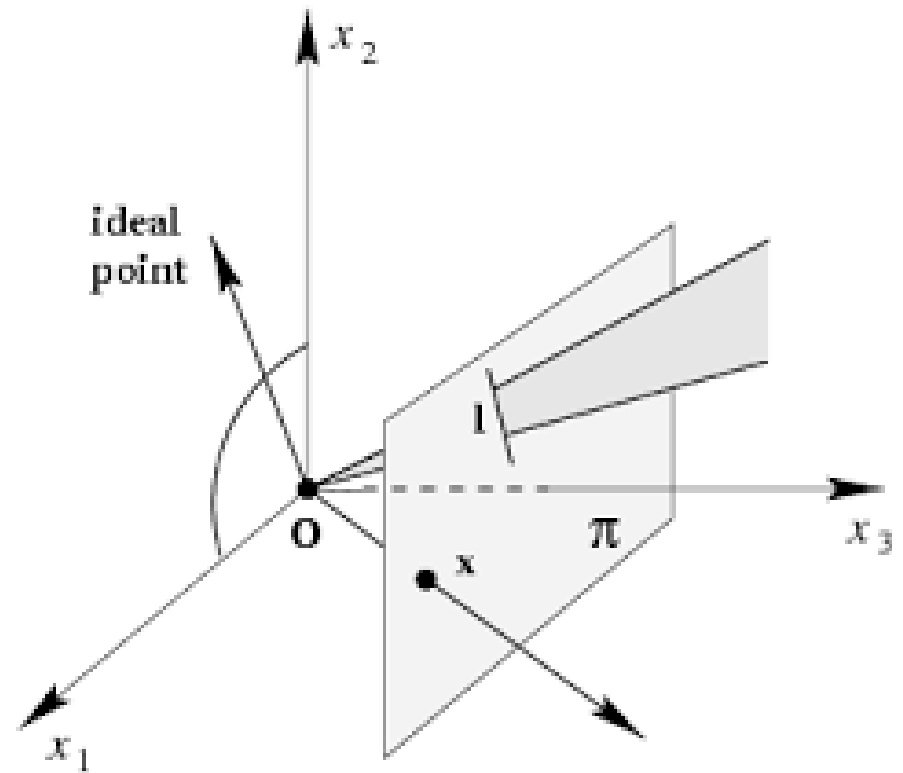
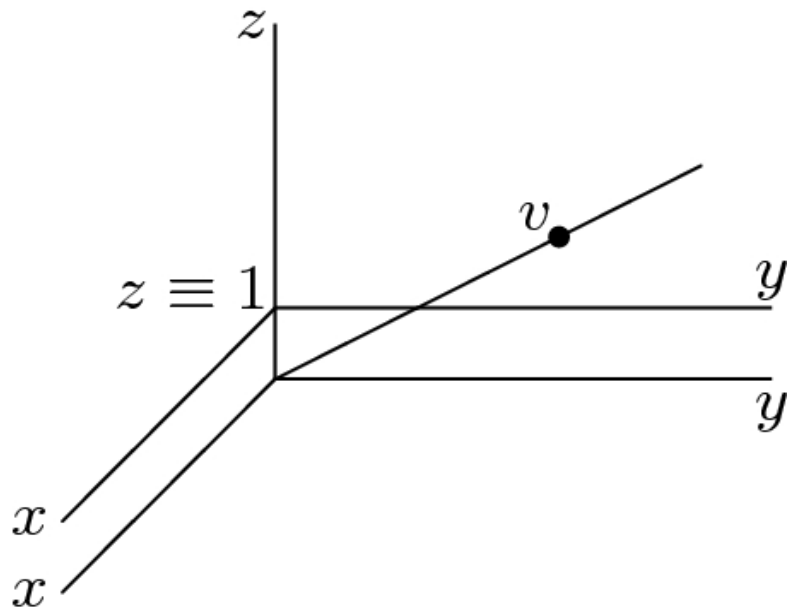


2D Primitives

Points and Lines in 2D

- A point (x, y) lies on a line (a, b, c) when:
 $ax+by+c = 0$ or $(a, b, c) (x, y, 1)^T = 0$
- Use homogeneous coordinates to represent points => add an extra coordinate
 - Note that scale is unimportant for determining incidence: $k(x, y, 1)$ is also on the line
 - Homogeneous coordinates (x_1, x_2, x_3) , but only two degrees of freedom
 - Equivalent to inhomogeneous coordinates (x, y)

Points and Lines in 2D



Points from Lines and Vice Versa

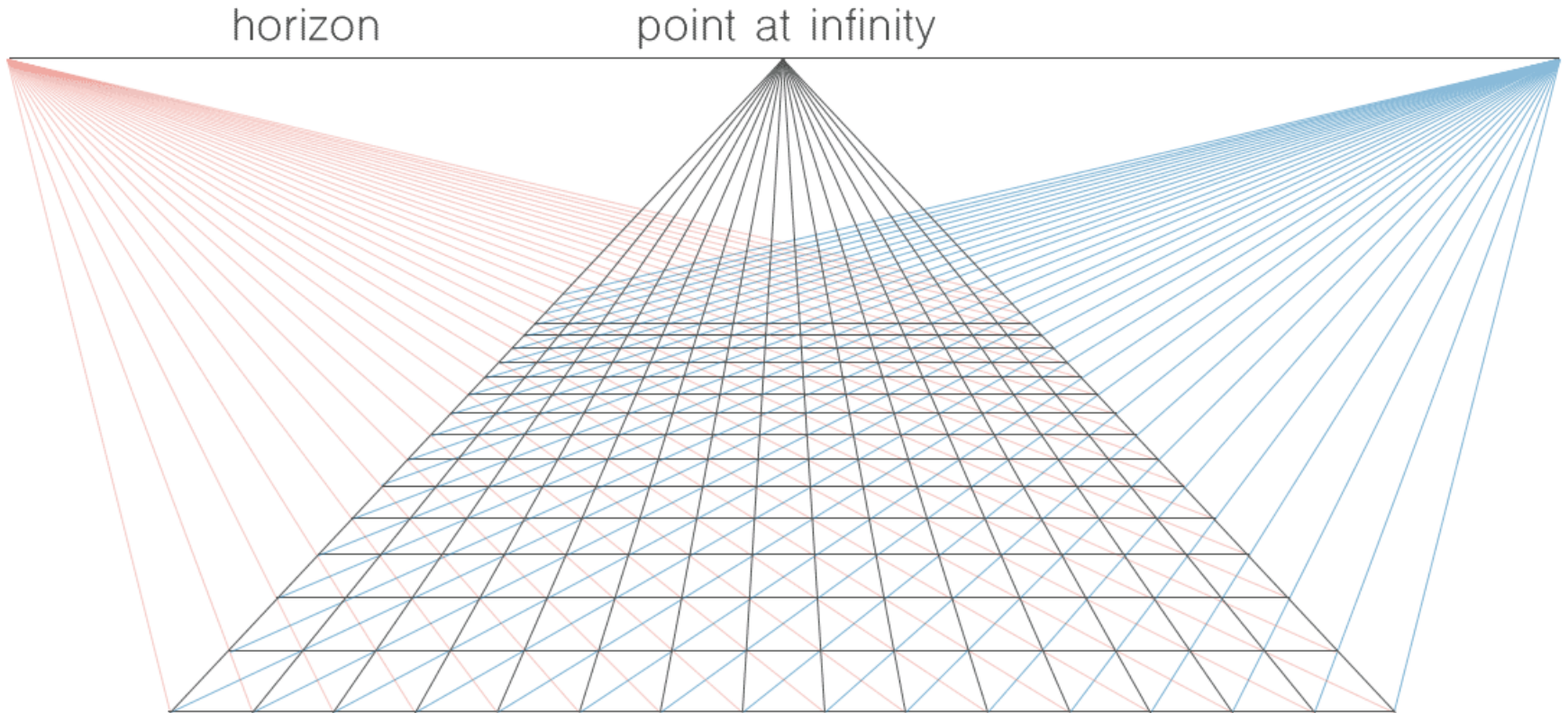
- The intersection of two lines l and l' is given by: $l \times l'$
- The line connecting two points x and x' is given by: $x \times x'$

$$\vec{C} = \vec{A} \times \vec{B} = \begin{pmatrix} A_x \\ A_y \\ A_z \end{pmatrix} \times \begin{pmatrix} B_x \\ B_y \\ B_z \end{pmatrix} = \begin{pmatrix} A_y \cdot B_z - A_z \cdot B_y \\ A_z \cdot B_x - A_x \cdot B_z \\ A_x \cdot B_y - A_y \cdot B_x \end{pmatrix}$$

Ideal Points and the Line at Infinity

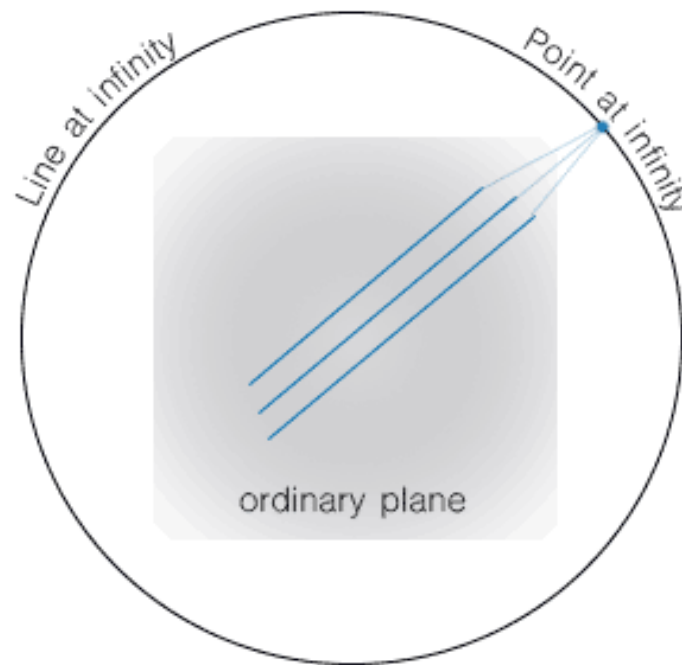


Ideal Points and the Line at Infinity



Ideal Points and the Line at Infinity

projective plane



Ideal Points and the Line at Infinity

- Intersection of two parallel lines:
 - $l = (a, b, c)$ and $l' = (a, b, c')$
 - $l \times l' = (b, -a, 0)$
- Ideal points: $(x_1, x_2, 0)$
- Belong to the line at infinity $l = (0, 0, 1)$
- $\mathbf{P}^2 = \mathbf{R}^3 - (0, 0, 0)$ (projective space)
 - In $\mathbf{P}^2$ there is no distinction between regular and ideal points

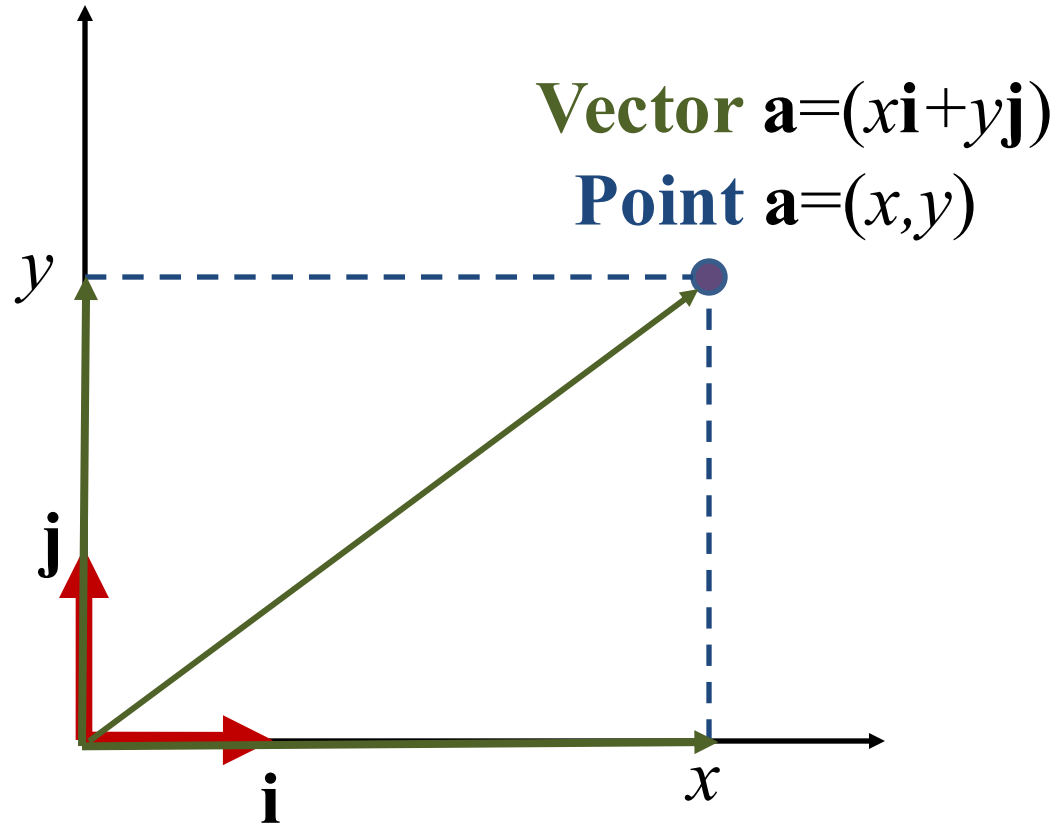
Rotation in 2D

- Matrices are operators that transform vectors

- 2D rotation matrix $R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$

- In homogeneous coordinates $\begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}$

2D Rotations



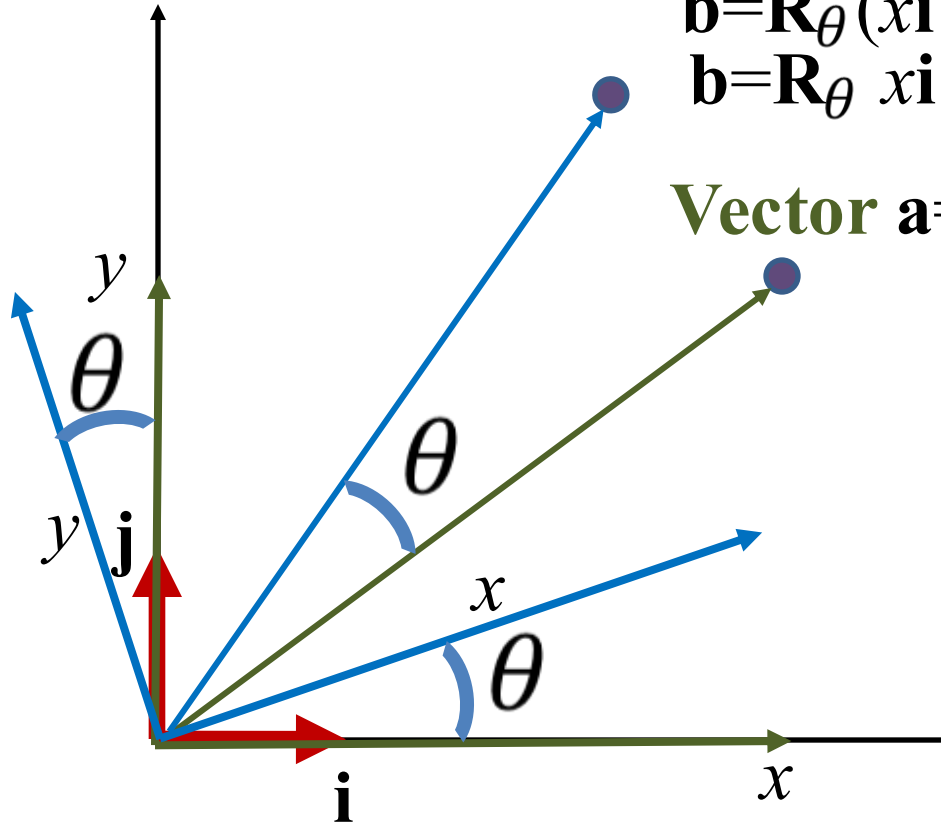
2D Rotations

Vector $\mathbf{b} = \mathbf{R}_\theta \mathbf{a}$

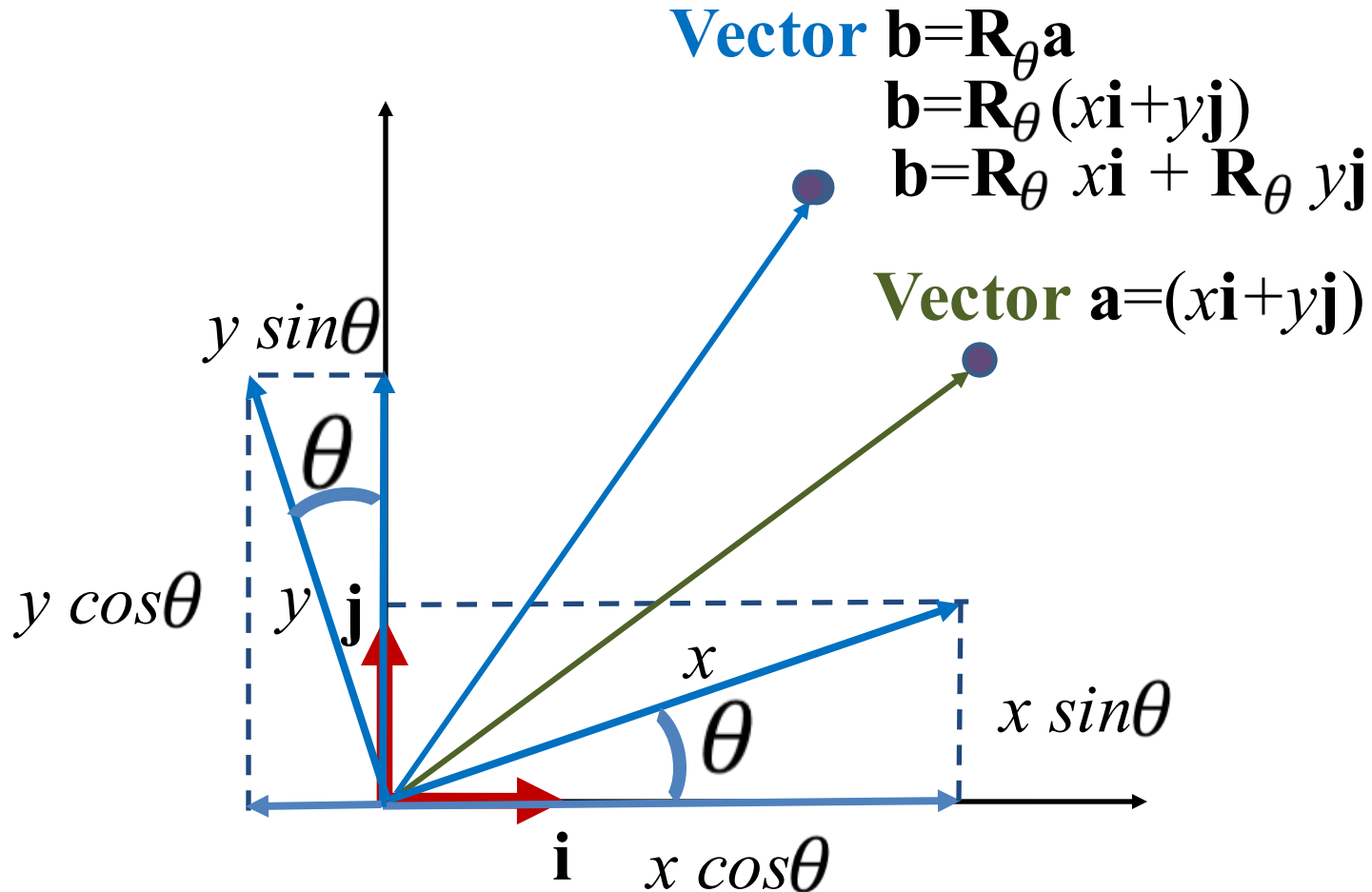
$$\mathbf{b} = \mathbf{R}_\theta (x\mathbf{i} + y\mathbf{j})$$

$$\mathbf{b} = \mathbf{R}_\theta x\mathbf{i} + \mathbf{R}_\theta y\mathbf{j}$$

Vector $\mathbf{a} = (x\mathbf{i} + y\mathbf{j})$

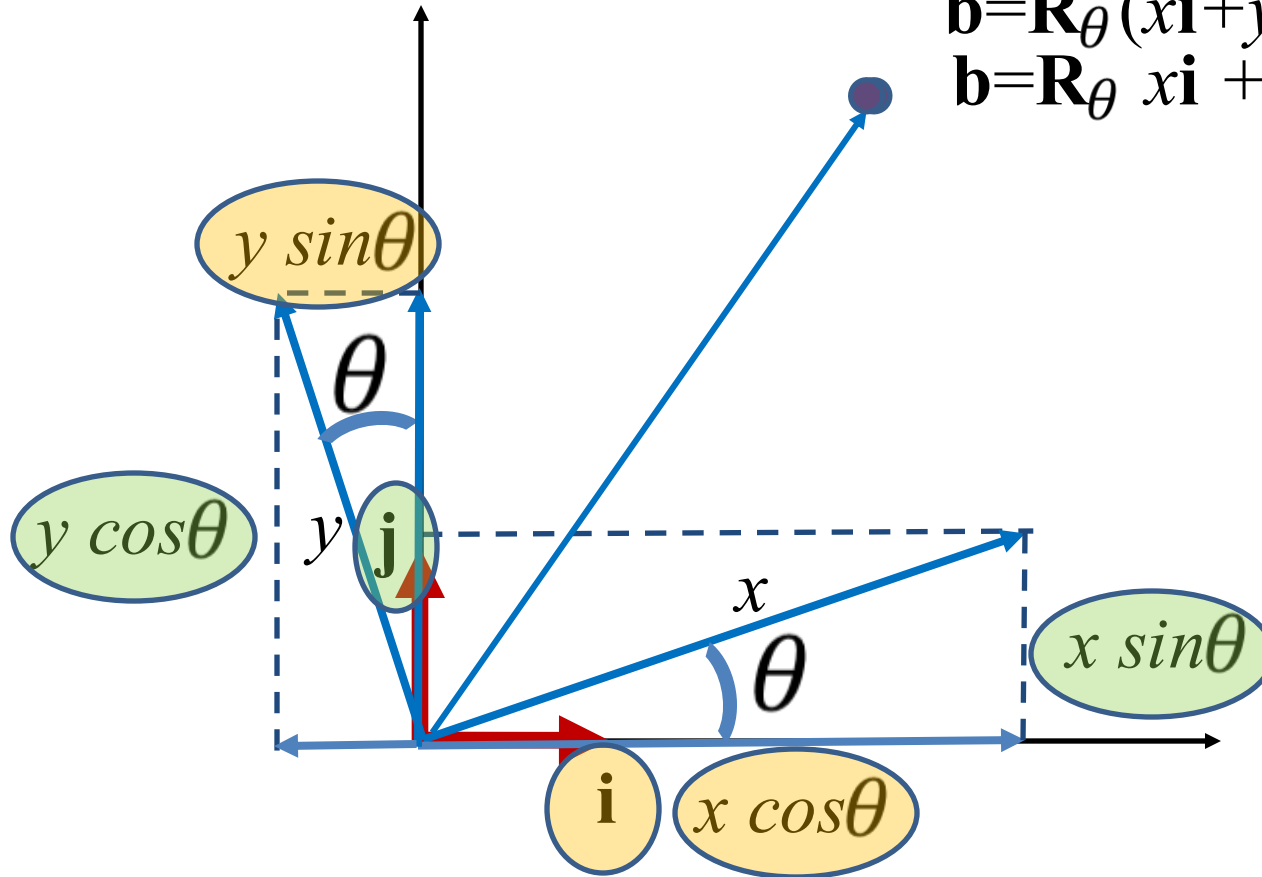


2D Rotations



2D Rotations

Vector $\mathbf{b} = \mathbf{R}_\theta \mathbf{a}$
 $\mathbf{b} = \mathbf{R}_\theta (x\mathbf{i} + y\mathbf{j})$
 $\mathbf{b} = \mathbf{R}_\theta x\mathbf{i} + \mathbf{R}_\theta y\mathbf{j}$

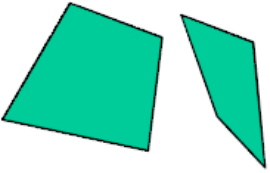
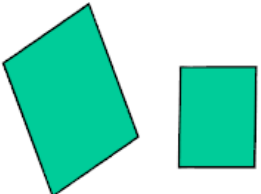
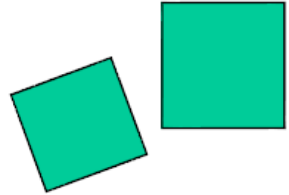
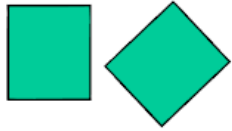


Hands-on: 2D Transformations

- How to translate a point in homogeneous and inhomogeneous coordinates?
- How to rotate a point around the origin?
- How to rotate a point around a center other than the origin?

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \quad \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Hierarchy of 2D Transformations

		transformed squares	invariants
Projective 8dof	$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$		Concurrency, collinearity, order of contact (intersection, tangency, inflection, etc.), cross ratio
Affine 6dof	$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		Parallellism, ratio of areas, ratio of lengths on parallel lines (e.g midpoints), linear combinations of vectors (centroids). The line at infinity l_∞
Similarity 4dof	$\begin{bmatrix} sr_{11} & sr_{12} & t_x \\ sr_{21} & sr_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		Ratios of lengths, angles. The circular points I, J
Euclidean 3dof	$\begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		lengths, areas.

Transformation of Points and Lines

Point transformation

$$\mathbf{x}' = \mathbf{H} \mathbf{x}$$

Line transformation

$$\mathbf{l}' = \mathbf{H}^{-T} \mathbf{l}$$

3D Primitives

3D points

3D point

$$(X, Y, Z)^{\top} \text{ in } \mathbf{R}^3$$

$$\mathbf{X} = (X_1, X_2, X_3, X_4)^{\top} \text{ in } \mathbf{P}^3$$

$$\mathbf{X} = \left(\frac{X_1}{X_4}, \frac{X_2}{X_4}, \frac{X_3}{X_4}, 1 \right)^{\top} = (X, Y, Z, 1)^{\top} \quad (X_4 \neq 0)$$

projective transformation

$$\mathbf{X}' = \mathbf{H} \mathbf{X} \quad (4 \times 4 - 1 = 15 \text{ dof})$$

Planes

3D plane

$$\pi_1 X + \pi_2 Y + \pi_3 Z + \pi_4 = 0$$

$$\pi_1 X_1 + \pi_2 X_2 + \pi_3 X_3 + \pi_4 X_4 = 0$$

$$\pi^\top X = 0$$

Transformation

$$X' = \mathbf{H} X$$

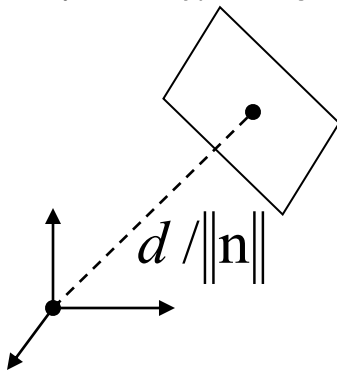
$$\pi' = \mathbf{H}^{-\top} \pi$$

Euclidean representation

$$\mathbf{n} \cdot \tilde{X} + d = 0 \quad \mathbf{n} = (\pi_1, \pi_2, \pi_3)^\top \quad \tilde{X} = (X, Y, Z)^\top$$

$$\pi_4 = d$$

$$X_4 = 1$$



Planes from points

Solve π from $X_1^\top \pi = 0$, $X_2^\top \pi = 0$ and $X_3^\top \pi = 0$

$$\begin{bmatrix} X_1^\top \\ X_2^\top \\ X_3^\top \end{bmatrix} \pi = \mathbf{0} \quad (\text{solve as right nullspace of } \pi) \quad \begin{bmatrix} X_1^\top \\ X_2^\top \\ X_3^\top \end{bmatrix}$$

Planes from points

Solve π from $X_1^\top \pi = 0$, $X_2^\top \pi = 0$ and $X_3^\top \pi = 0$

$$\begin{bmatrix} X_1^\top \\ X_2^\top \\ X_3^\top \end{bmatrix} \pi = 0 \quad (\text{solve as right nullspace of } \begin{bmatrix} X_1^\top \\ X_2^\top \\ X_3^\top \end{bmatrix})$$

Or implicitly from coplanarity condition

$$\det \begin{bmatrix} X_1 & (X_1)_x & (X_2)_x & (X_3)_x \\ X_2 & (X_1)_y & (X_2)_y & (X_3)_y \\ X_3 & (X_1)_z & (X_2)_z & (X_3)_z \\ X_4 & (X_1)_w & (X_2)_w & (X_3)_w \end{bmatrix} = 0$$

$$X_1 D_{234} - X_2 D_{134} + X_3 D_{124} - X_4 D_{123} = 0$$

$$\pi = (D_{234}, -D_{134}, D_{124}, -D_{123})^\top$$

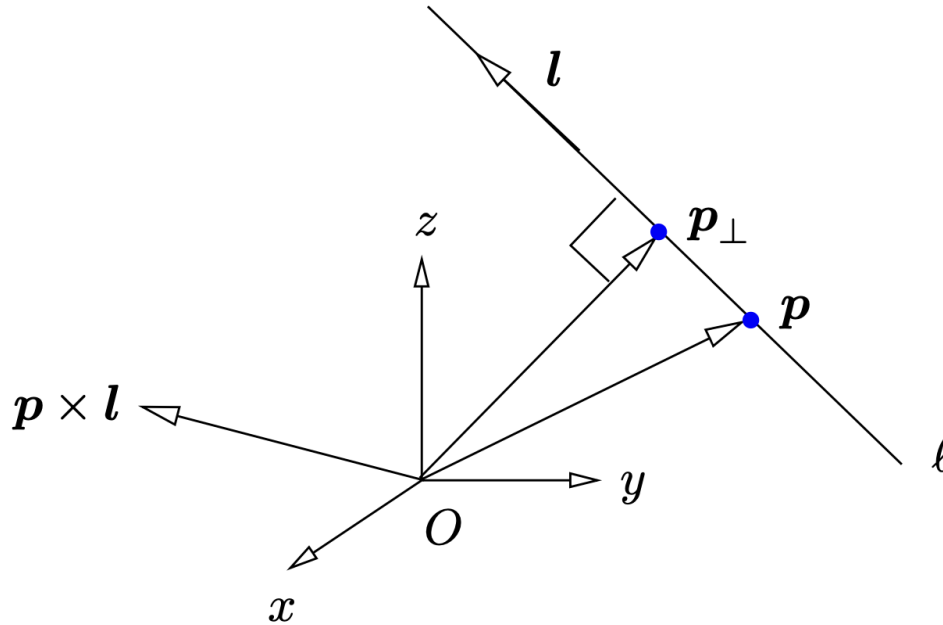
Points from planes

Solve X from $\pi_1^\top X = 0$, $\pi_2^\top X = 0$ and $\pi_3^\top X = 0$

$$\begin{bmatrix} \pi_1^\top \\ \pi_2^\top \\ \pi_3^\top \end{bmatrix} X = 0 \quad (\text{solve as right nullspace of } X) \quad \begin{bmatrix} \pi_1^\top \\ \pi_2^\top \\ \pi_3^\top \end{bmatrix}$$

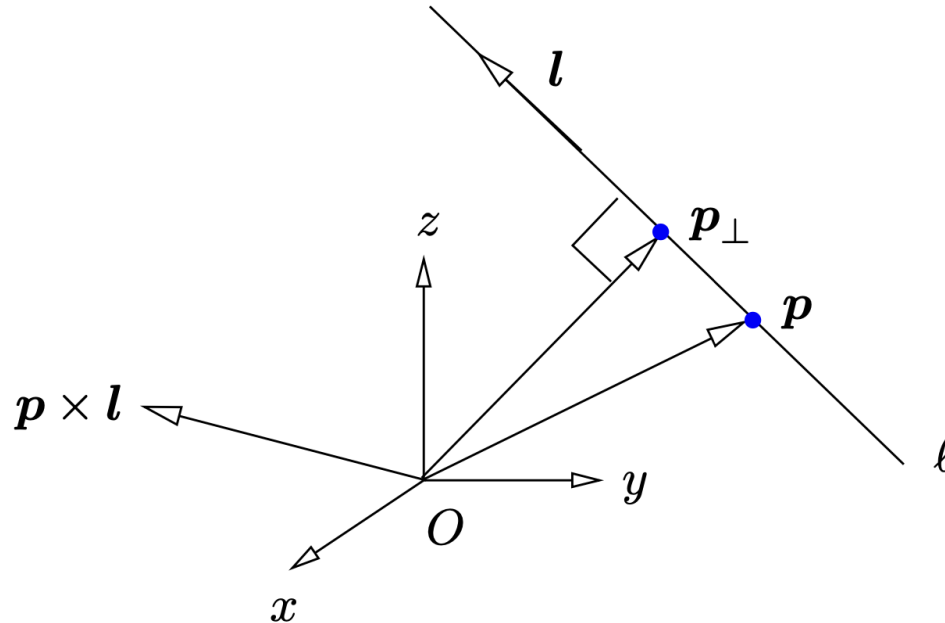
Lines are complicated (**they live in P^5**)...

Plucker Lines



Plücker coordinates $(\ell, p \times \ell)$ of a line. Here $p_{\perp}$ is the closest point on the line to the origin O .

Plucker Lines



Plücker coordinates $(l, p \times l)$ of a line. Here $p_{\perp}$ is the closest point on the line to the origin O .

Reciprocal Product

$$(\hat{l}_1, m_1) * (\hat{l}_2, m_2) \stackrel{\text{def}}{=} \hat{l}_1 \cdot m_2 + \hat{l}_2 \cdot m_1.$$

Distance Between Lines Two lines ℓ_1 and ℓ_2 , in Plücker coordinates (l_1, m_1) and (l_2, m_2) , have the distance

$$d = \begin{cases} \frac{|(l_1, m_1) * (l_2, m_2)|}{\|l_1 \times l_2\|}, & \text{if } l_1 \times l_2 \neq 0; \\ \frac{\|l_1 \times (m_1 - m_2/s)\|}{\|l_1\|^2}, & \text{otherwise, where } l_2 = sl_1 \text{ for some } s \neq 0. \end{cases} \quad (10)$$

Rotations

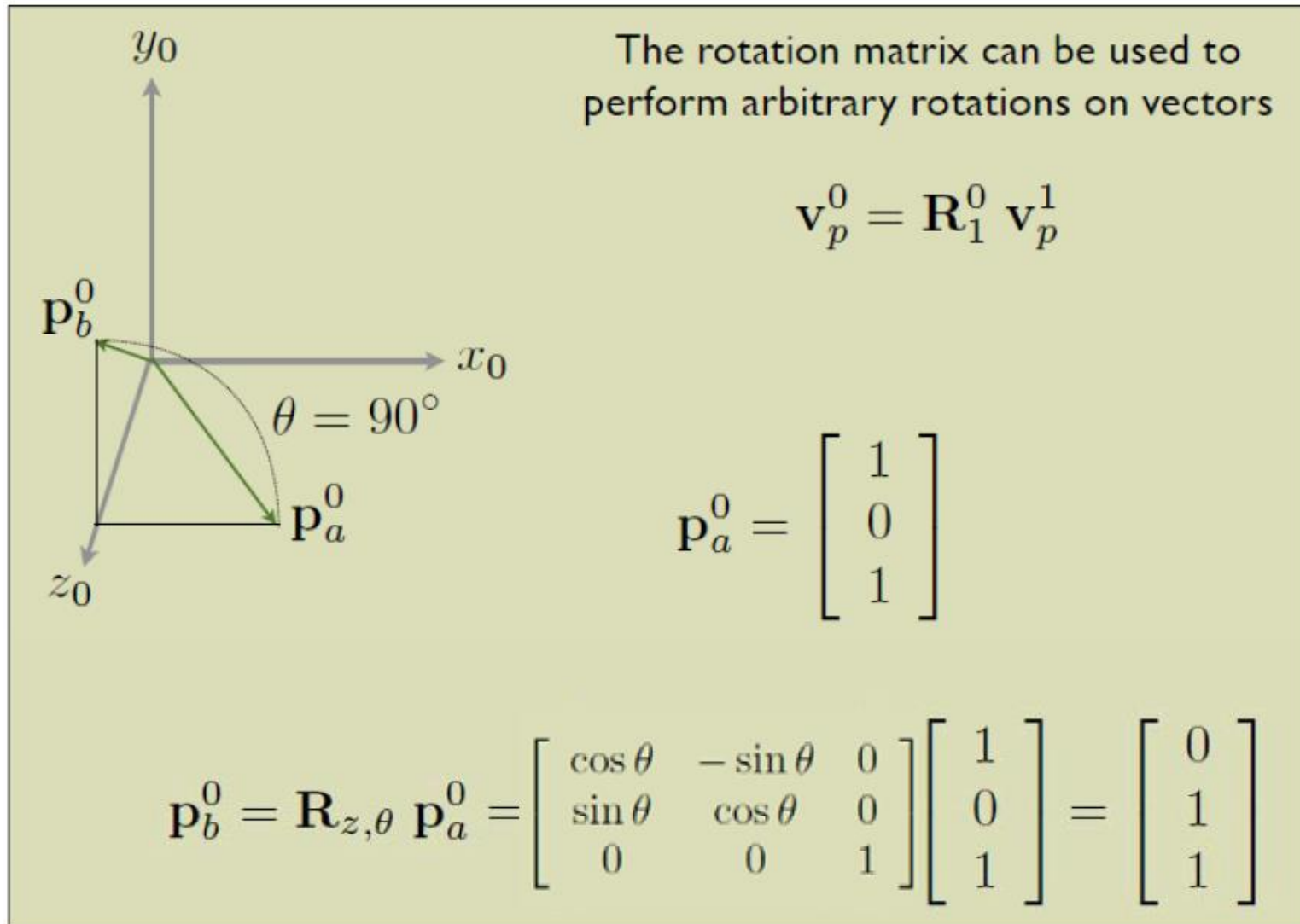
- Rotation matrices around the 3 axes
=> What is the inverse of a rotation matrix?

$$\mathbf{R}_{x,\theta} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

$$\mathbf{R}_{y,\theta} = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

$$\mathbf{R}_{z,\theta} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Rotation Example



Parameterization of Rotations

- In 3D, the 9-element rotation matrix has 3 DOF
- Several methods exist for representing a 3D rotation
 - Euler angles
 - Pitch, Roll, Yaw angles
 - Axis/Angle representation
 - Quaternions

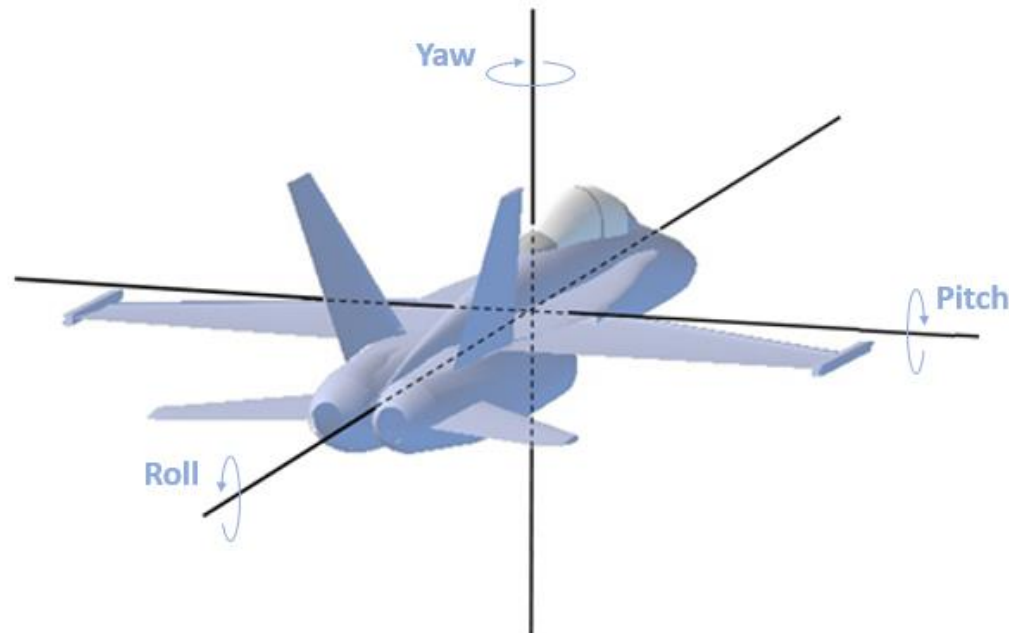
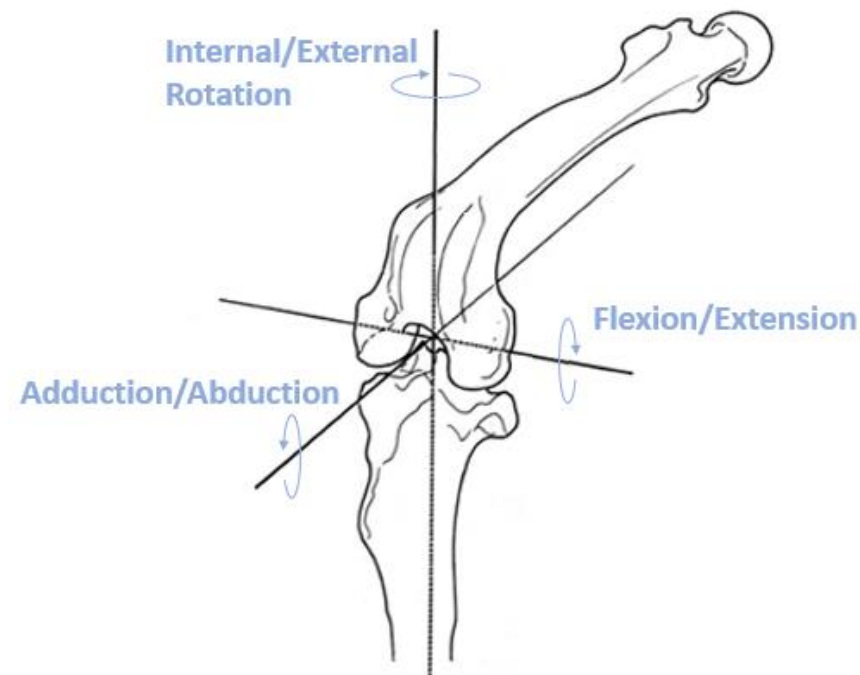
Rotation are compositional

- Three consecutive elemental rotations (rotations about the axes of a coordinate system) are sufficient to reach any target frame.
- But which coordinate systems do we use?
- “Global” vs “Body” Coordinate Systems

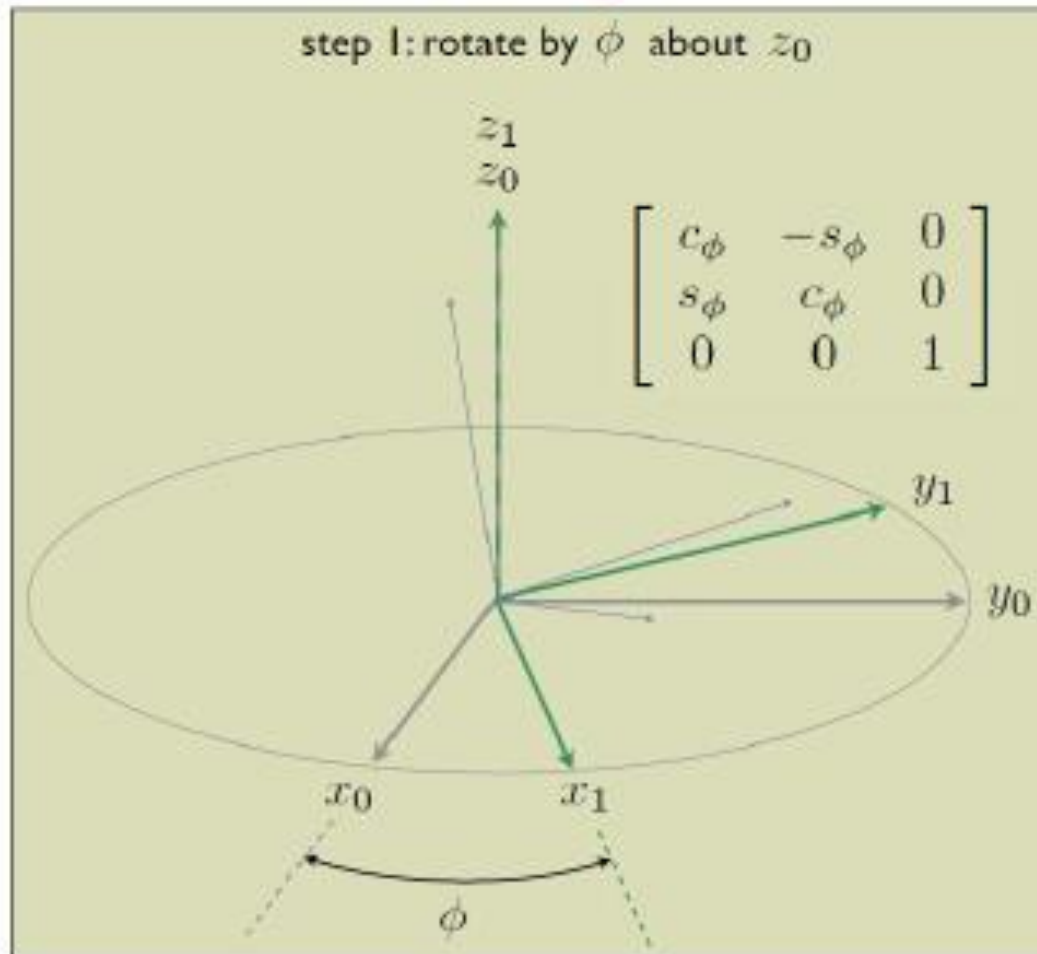
BodyXYZ	BodyYZX	BodyZXY	BodyXZY	BodyYXZ	BodyZYX
BodyXYX	BodyXZX	BodyYXY	BodyYZY	BodyZXZ	BodyZYZ
GlobalXYZ	GlobalYZX	GlobalZXY	GlobalXZY	GlobalYXZ	GlobalZYX
GlobalXYX	GlobalXZX	GlobalYXY	GlobalYZY	GlobalZXZ	GlobalZYZ

Euler Angles

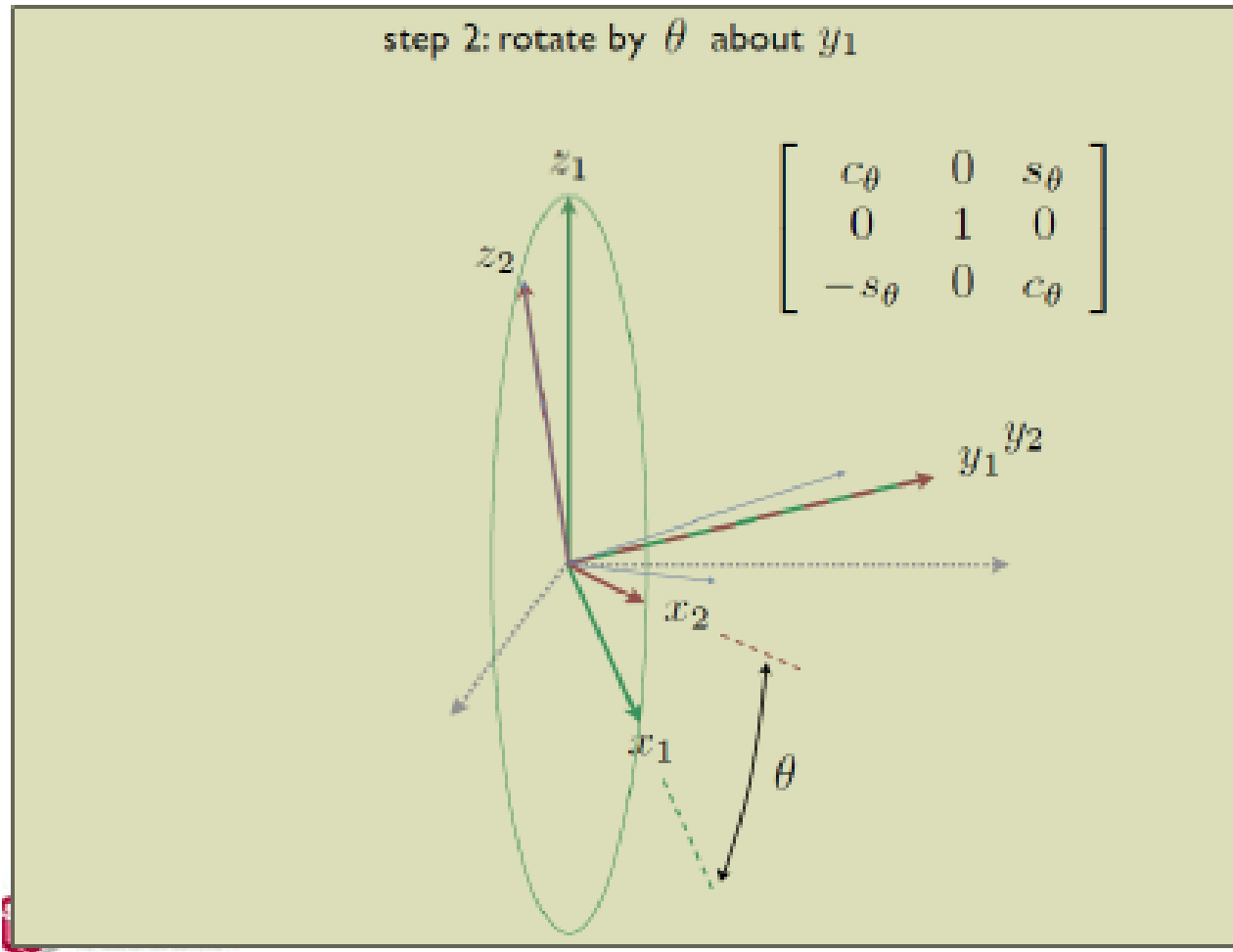
- Define in Object-based coordinate systems



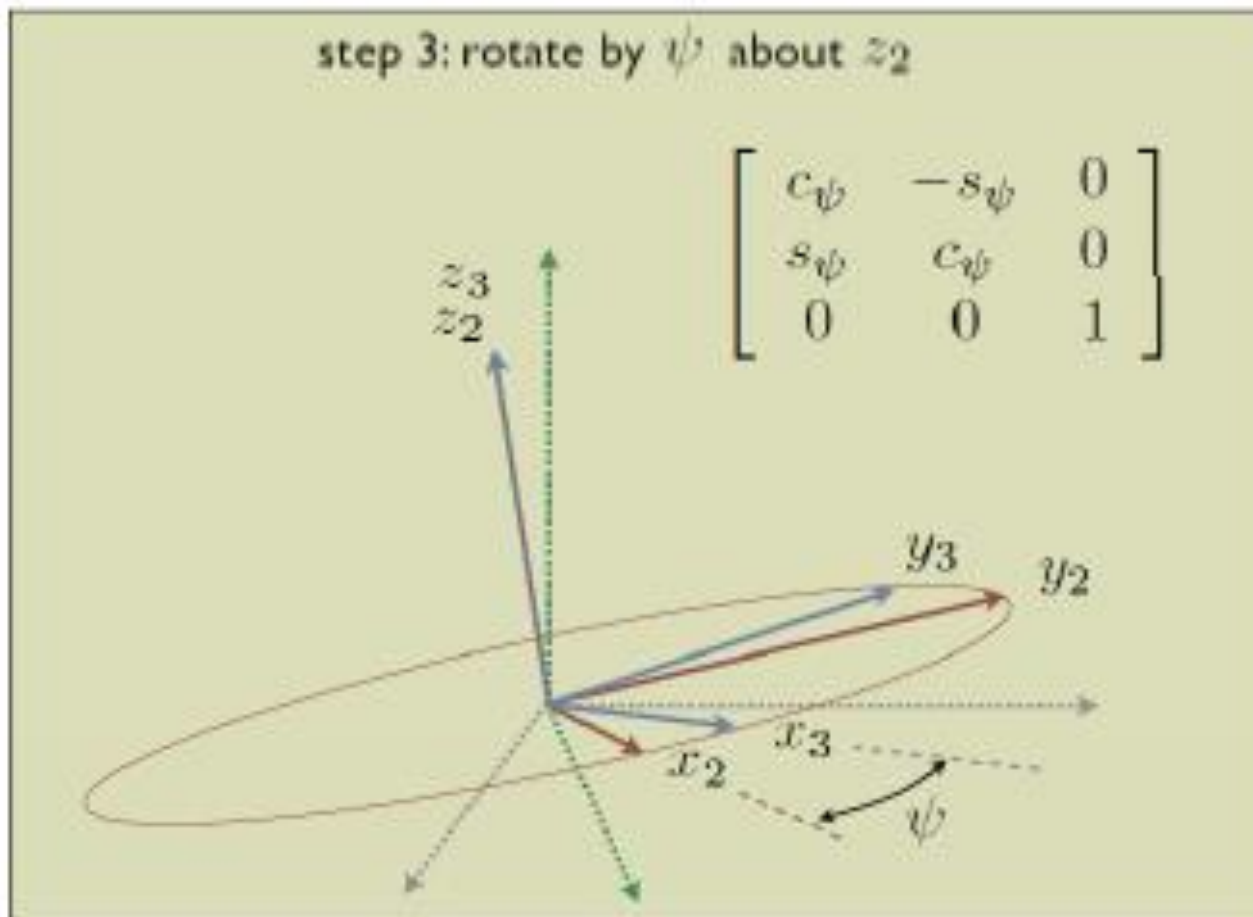
Euler Angles



Euler Angles



Euler Angles



Euler Angles to Rotation Matrix

(**post**-multiply using the **basic rotation matrices**)

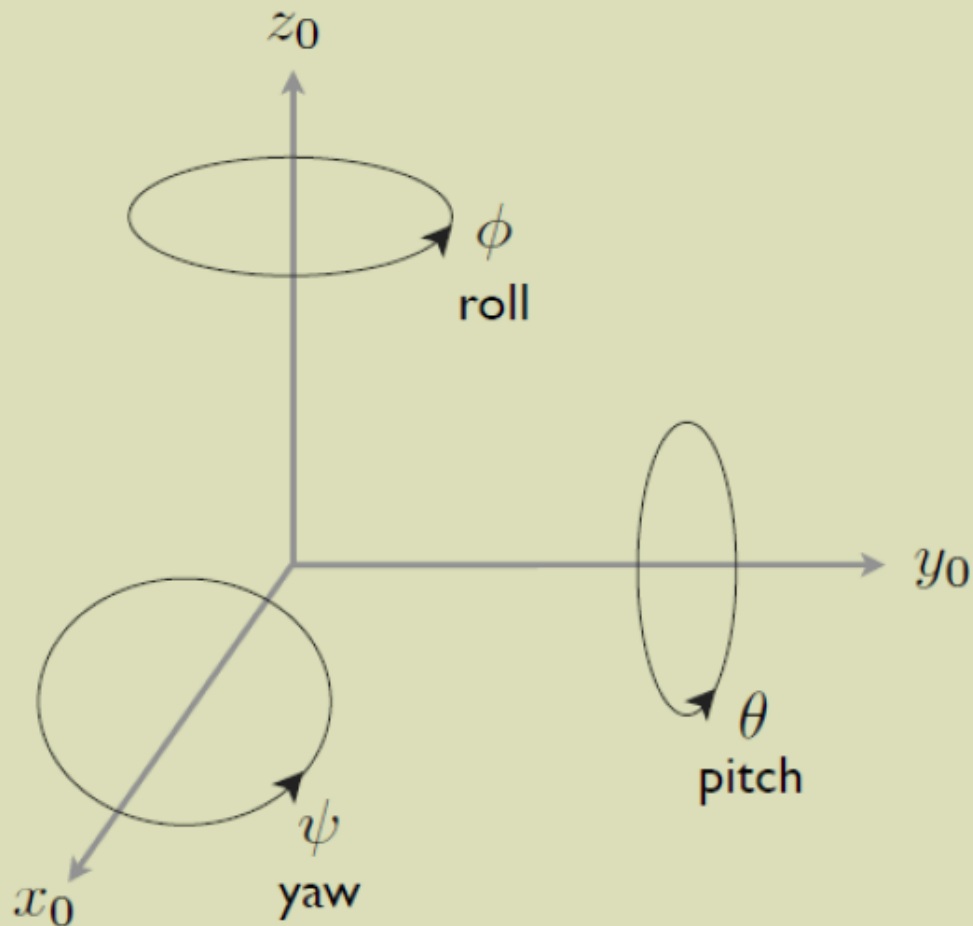
$$\mathbf{R} = \mathbf{R}_{z,\phi} \mathbf{R}_{y,\theta} \mathbf{R}_{z,\psi}$$

$$= \begin{bmatrix} c_\phi & -s_\phi & 0 \\ s_\phi & c_\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix} \begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} c_\phi c_\theta c_\psi - s_\phi s_\psi & -c_\phi c_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta \\ s_\phi c_\theta c_\psi + c_\phi s_\psi & -s_\phi c_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta \\ -s_\theta c_\psi & s_\theta s_\psi & c_\theta \end{bmatrix}$$

Roll, Pitch, Yaw Angles

defined as a set of three angles about a **fixed** reference



Roll, Pitch, Yaw Angles to Rotation Matrix

(**pre**-multiply using the **basic rotation matrices**)

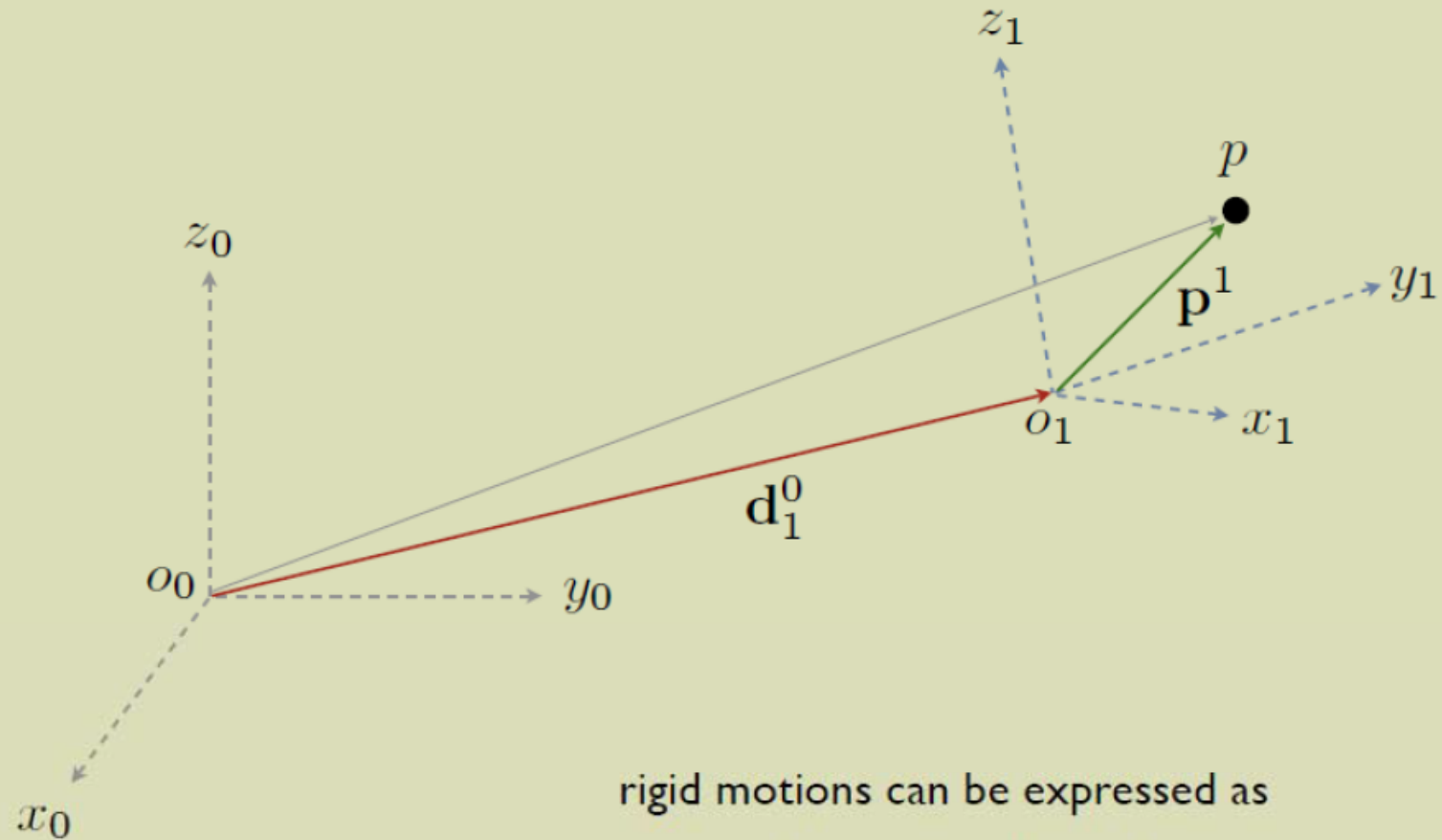
$$\mathbf{R} = \mathbf{R}_{z,\phi} \mathbf{R}_{y,\theta} \mathbf{R}_{x,\psi}$$

$$= \begin{bmatrix} c_\phi & -s_\phi & 0 \\ s_\phi & c_\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\psi & -s_\psi \\ 0 & s_\psi & c_\psi \end{bmatrix}$$

$$= \begin{bmatrix} c_\phi c_\theta & c_\phi s_\theta s_\psi - s_\phi c_\psi & s_\phi s_\psi + c_\phi s_\theta c_\psi \\ s_\phi c_\theta & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{bmatrix}$$

Rigid Motion

a **rigid motion** couples pure translation with pure rotation



$$p^0 = R_1^0 p^1 + d_1^0$$

Homogeneous Transformation

a **homogeneous transform** is a matrix representation of rigid motion, defined as

$$\mathbf{H} = \begin{bmatrix} \mathbf{R} & \mathbf{d} \\ \mathbf{0} & 1 \end{bmatrix}$$

where $\mathbf{R}$ is the 3x3 rotation matrix, and $\mathbf{d}$ is the 1x3 translation vector

$$\mathbf{H} = \begin{bmatrix} n_x & s_x & a_x & d_x \\ n_y & s_y & a_y & d_y \\ n_z & s_z & a_z & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

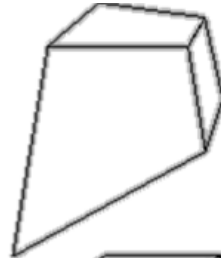
the **inverse** of a homogeneous transform can be expressed as

$$\mathbf{H}^{-1} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{d} \\ 0 & 1 \end{bmatrix}$$

Hierarchy of 3D Transformations

Projective
15dof

$$\begin{bmatrix} A & t \\ v^T & v \end{bmatrix}$$



Intersection and tangency

Affine
12dof

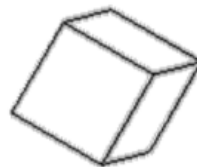
$$\begin{bmatrix} A & t \\ 0^T & 1 \end{bmatrix}$$



Parallellism of planes,
Volume ratios, centroids,
The plane at infinity π_∞

Similarity
7dof

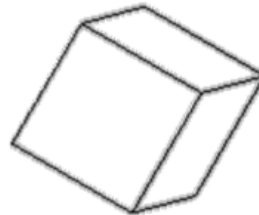
$$\begin{bmatrix} sR & t \\ 0^T & 1 \end{bmatrix}$$



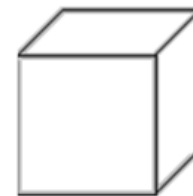
Angles, ratios of length
The absolute conic Ω_∞

Euclidean
6dof

$$\begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix}$$

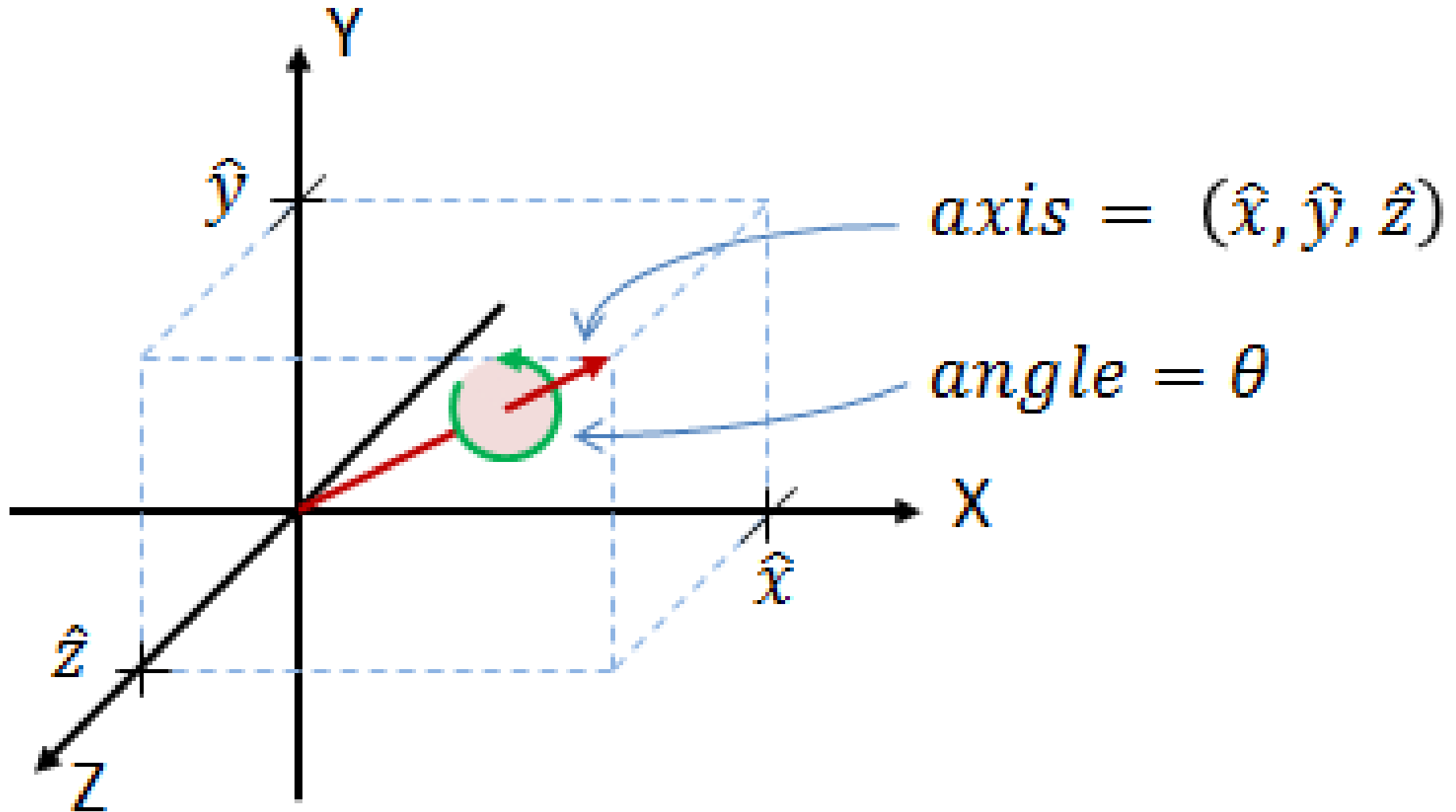


Volume



ETH

Rotation angle-axis



Quaternions

Quaternion Representation

$$\mathbf{q} = (q_0, q_1, q_2, q_3)$$

where:

$$q_0 = \cos\left(\frac{\theta}{2}\right)$$

$$q_1 = \hat{x} \sin\left(\frac{\theta}{2}\right)$$

$$q_2 = \hat{y} \sin\left(\frac{\theta}{2}\right)$$

$$q_3 = \hat{z} \sin\left(\frac{\theta}{2}\right)$$

Quaternion Use

Transform point into quaternion

$$\mathbf{p} = (p_0, p_1, p_2, p_3) = (0, x, y, z)$$

Rotation is now Quaternion Multiplication

$$\mathbf{p}' = \mathbf{q}^{-1} \mathbf{p} \mathbf{q}$$

Where multiplication is defined as

$$q_1 = (w_1, \mathbf{v}_1), \quad q_2 = (w_2, \mathbf{v}_2)$$

$$q_1 q_2 = (w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$$

And quaternion inversions is defined as

$$\mathbf{q}^{-1} = (q_0, -q_1, -q_2, -q_3)$$

Quaternions



- Alternative to Euler Angles
- Developed by Sir William Rowan Hamilton [1843]
- Quaternions are 4-D complex numbers
$$\mathbf{q} = q_0 + iq_1 + jq_2 + kq_3$$
- With one real axis
- And three imaginary axes: i, j, k



TL;DL It turns out that quaternions are effectively an angle-axis representation of an orientation...just think of them as a way of encoding that information

Hamilton Math Inst.,
Trinity College

Quaternions

- Introduced to Computer Graphics by Shoemake [1985]
- Given an angle and axis, easy to convert to and from quaternion
 - Euler angle conversion to and from arbitrary axis and angle difficult
- Quaternions allow stable and constant interpolation of orientations
 - Cannot be done easily with Euler angles

Unit Quaternions

- For convenience, we will use only unit length quaternions
- These correspond to the set of 4D vectors

$$|\mathbf{q}| = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2} = 1$$

- They form the ‘surface’ of a 4D hypersphere of radius 1

Quaternions as Rotations

- A quaternion can represent a rotation by angle θ around a unit vector $\mathbf{a}$:

$$\mathbf{q} = \begin{bmatrix} \cos \frac{\theta}{2} & a_x \sin \frac{\theta}{2} & a_y \sin \frac{\theta}{2} & a_z \sin \frac{\theta}{2} \end{bmatrix}$$

or

$$\mathbf{q} = \left\langle \cos \frac{\theta}{2}, \mathbf{a} \sin \frac{\theta}{2} \right\rangle$$

- If $\mathbf{a}$ is unit length, then $\mathbf{q}$ will be also

Quaternion to Matrix

- To convert a quaternion to a rotation matrix:

$$\begin{bmatrix} 1 - 2q_2^2 - 2q_3^2 & 2q_1q_2 + 2q_0q_3 & 2q_1q_3 - 2q_0q_2 \\ 2q_1q_2 - 2q_0q_3 & 1 - 2q_1^2 - 2q_3^2 & 2q_2q_3 + 2q_0q_1 \\ 2q_1q_3 + 2q_0q_2 & 2q_2q_3 - 2q_0q_1 & 1 - 2q_1^2 - 2q_2^2 \end{bmatrix}$$

Matrix to Quaternion

- Matrix to quaternion is not hard
 - It involves a few ‘if’ statements,
 - a square root,
 - three divisions,
 - and some other stuff
- $\text{tr}(\mathbf{M})$ is the trace
 - sum of the diagonal elements

$$q_0 = \frac{1}{2} \sqrt{\text{tr}(\mathbf{M})}$$

$$q_1 = \frac{m_{21} - m_{12}}{4q_0}$$

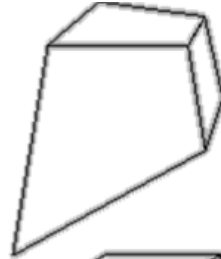
$$q_2 = \frac{m_{02} - m_{20}}{4q_0}$$

$$q_3 = \frac{m_{10} - m_{01}}{4q_0}$$

Hierarchy of 3D Transformations

Projective
15dof

$$\begin{bmatrix} A & t \\ v^T & v \end{bmatrix}$$



Intersection and tangency

Affine
12dof

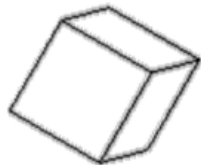
$$\begin{bmatrix} A & t \\ 0^T & 1 \end{bmatrix}$$



Parallellism of planes,
Volume ratios, centroids,
The plane at infinity π_∞

Similarity
7dof

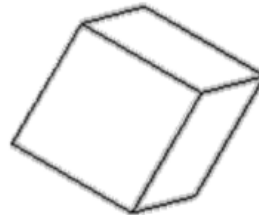
$$\begin{bmatrix} sR & t \\ 0^T & 1 \end{bmatrix}$$



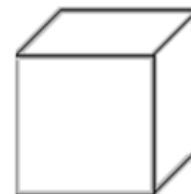
Angles, ratios of length
The absolute conic Ω_∞

Euclidean
6dof

$$\begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix}$$



Volume



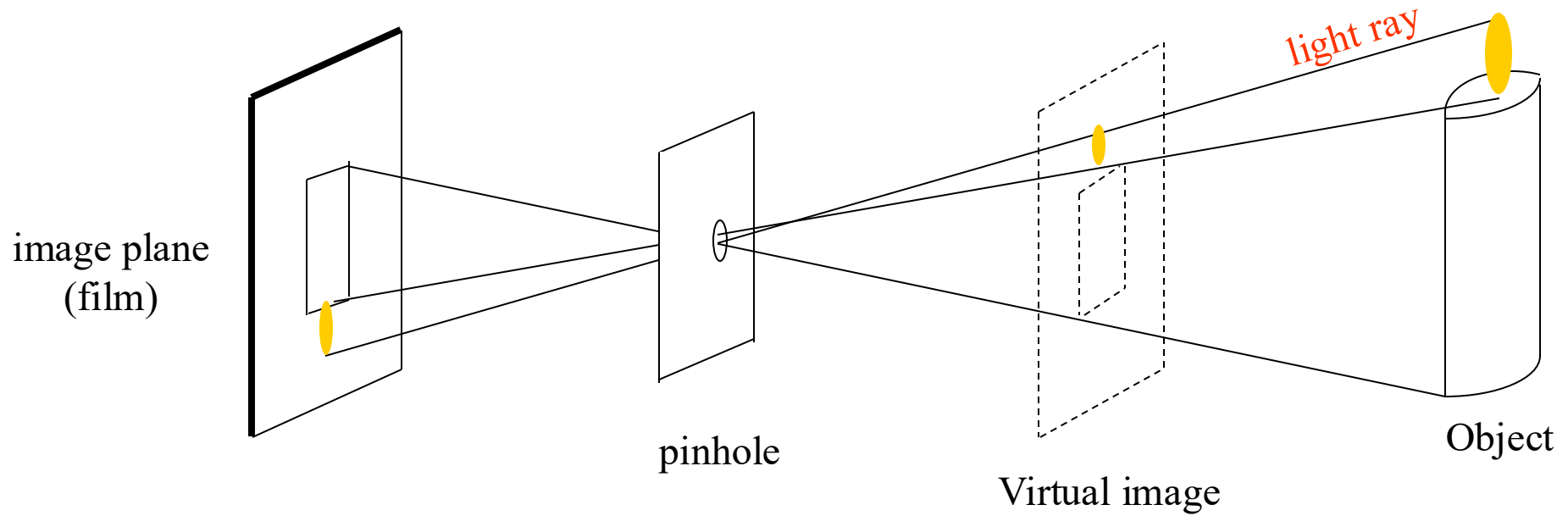
ETH

Image Formation

Based on slides by John Oliensis

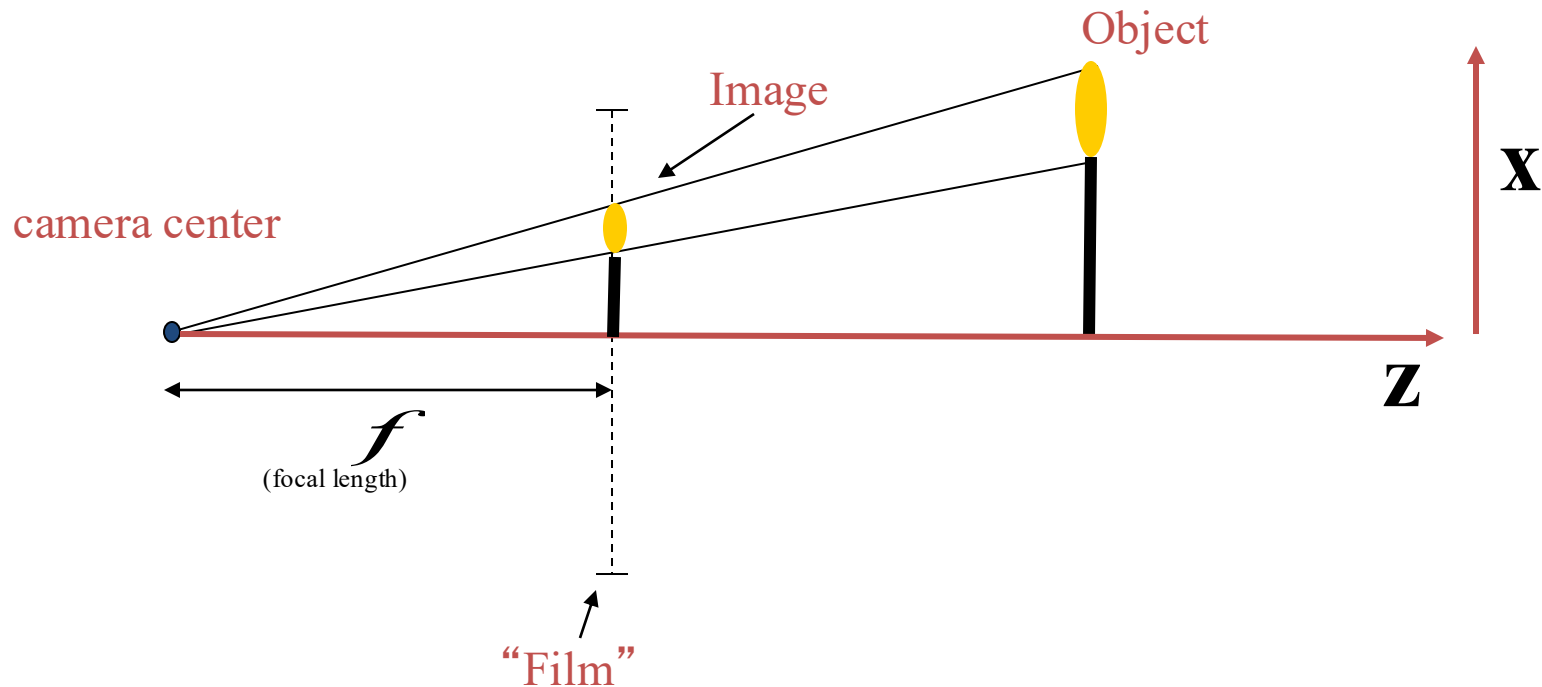
Image Formation

Pinhole camera

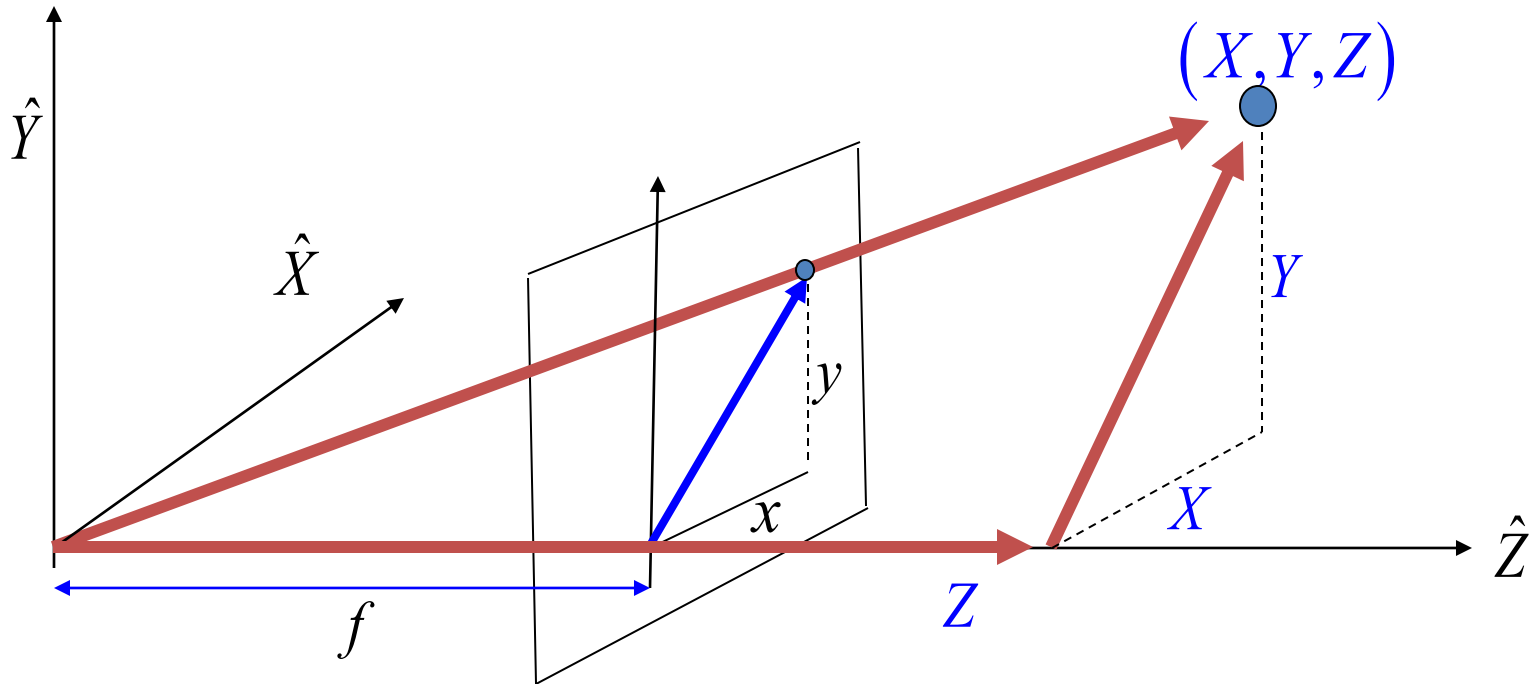


Projection Equation

- 2D world $\rightarrow$ 1D image



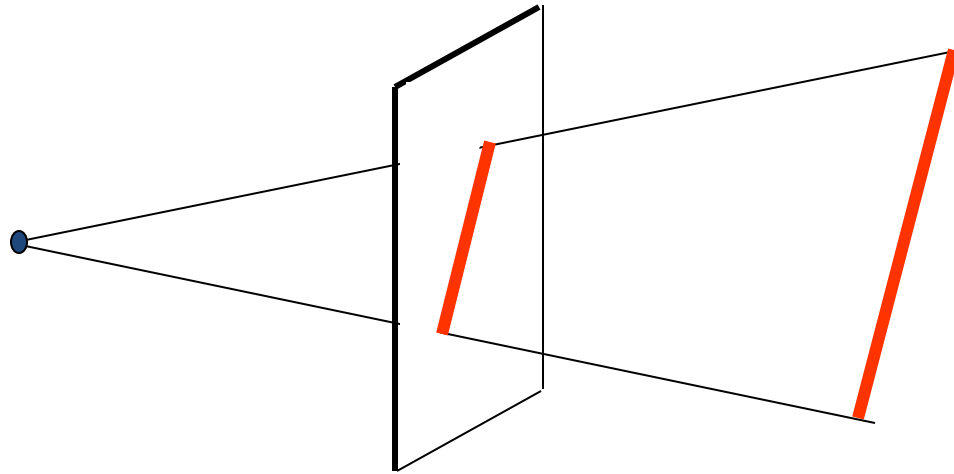
Projection Equation: 3D



Similar triangles: $\frac{x}{X} = \frac{y}{Y} = \frac{f}{Z} \longrightarrow \boxed{(x, y) = \frac{f}{Z}(X, Y)}$

Perspective Projection: Properties

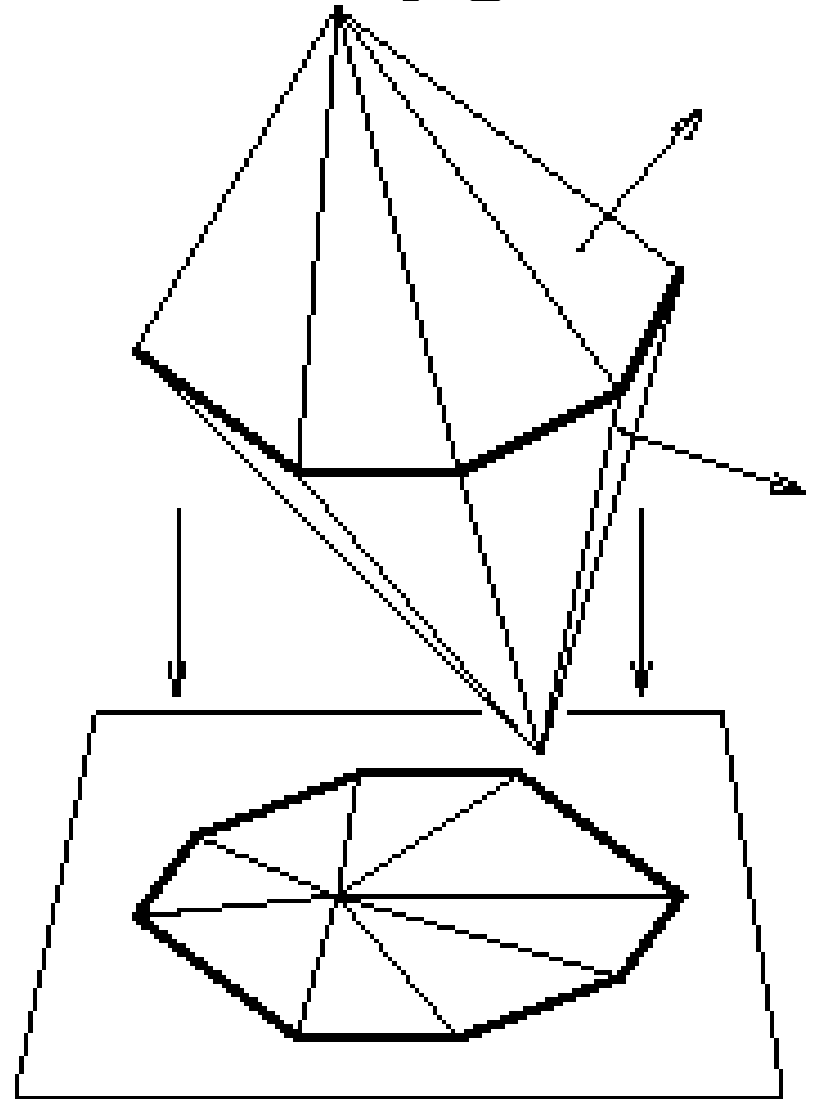
- 3D points $\rightarrow$ image points
- 3D straight lines $\rightarrow$ image straight lines



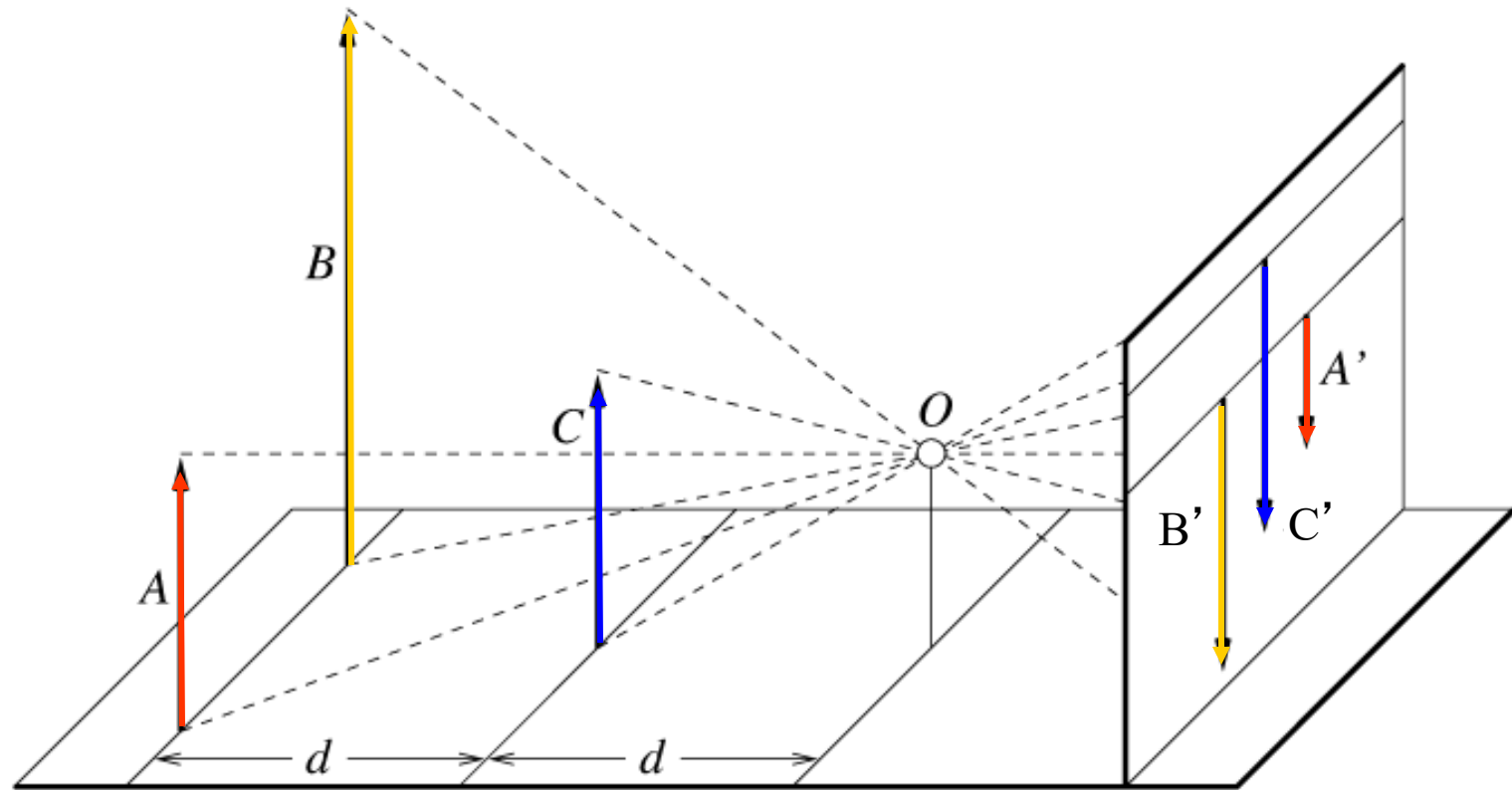
- 3D Polygons $\rightarrow$ image polygons

Polyhedra Project to Polygons

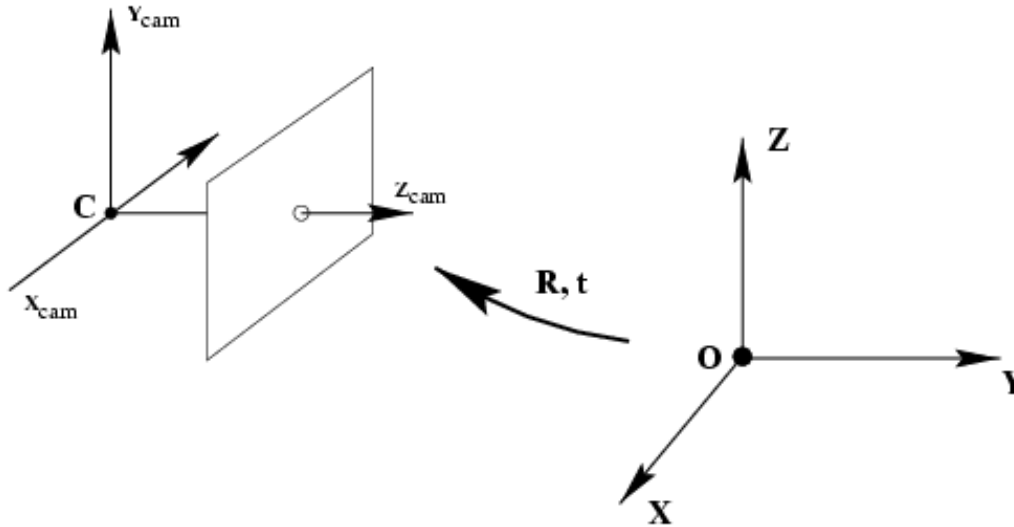
(since lines project to lines)



Properties: Distant objects are smaller



Extrinsic Parameters



Scene motion

$$M = \begin{bmatrix} R_{(3 \times 3)} & t_{(3 \times 1)} \\ 0_{(1 \times 3)} & 1 \end{bmatrix}$$

Camera motion

$$M' = \begin{bmatrix} R^T_{(3 \times 3)} & -(R^T t)_{3 \times 1} \\ 0_{(1 \times 3)} & 1 \end{bmatrix}$$

Projection matrix

- Includes coordinate transformation and camera intrinsic parameters

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

- Everything we need to know about a pinhole camera
- Unambiguous
- Can be decomposed into parameters

Projection matrix

- Mapping from 2-D to 3-D is a function of internal and external parameters

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R^\top & | & -R^\top t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\lambda x = K \begin{bmatrix} R^\top & | & -R^\top t \end{bmatrix} X$$

$$\lambda x = PX$$

Hands On: Camera Motion

- Choose a few 3D points visible to a camera at the origin. ($f=500$, $w=500$, $h=500$)
- Now, move the camera by 2 units of length on the z axis. What happens to the images of the points?
- Rotate the points by 45 degrees about the z axis of the camera and then translate them by 5 units on the z axis away from the camera. What are the new images of the points?

Projective Transformations in 2D

Definition:

A *projectivity* is an invertible mapping h from P^2 to itself such that three points x_1, x_2, x_3 lie on the same line if and only if $h(x_1), h(x_2), h(x_3)$ do.

Theorem:

A mapping $h: P^2 \rightarrow P^2$ is a projectivity if and only if there exist a non-singular 3×3 matrix $\mathbf{H}$ such that for any point in P^2 represented by a vector $\mathbf{x}$ it is true that $h(\mathbf{x}) = \mathbf{H}\mathbf{x}$

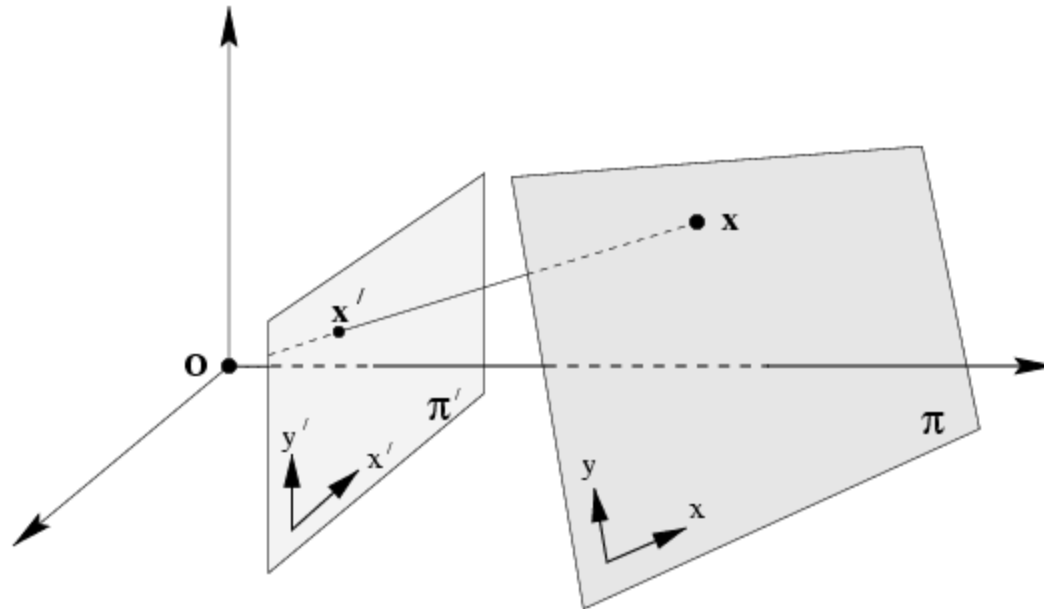
Definition: Projective transformation

$$\begin{pmatrix} x'_1 \\ x'_2 \\ x'_3 \end{pmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \quad \text{or} \quad \mathbf{x}' = \mathbf{H} \mathbf{x}$$

8DOF

projectivity=collineation=projective transformation=homography

Mapping between planes



central projection may be expressed by $x' = Hx$
(application of theorem)

Removing Projective Distortion



select four points in a plane with known coordinates

$$x' = \frac{x'_1}{x'_3} = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}} \quad y' = \frac{x'_2}{x'_3} = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

$$\begin{aligned} x'(h_{31}x + h_{32}y + h_{33}) &= h_{11}x + h_{12}y + h_{13} \\ y'(h_{31}x + h_{32}y + h_{33}) &= h_{21}x + h_{22}y + h_{23} \end{aligned} \quad (\text{linear in } h_{ij})$$

(2 constraints/point, 8DOF $\Rightarrow$ 4 points needed)

Remarks: no calibration at all necessary,
better ways to compute (see later)

A Hierarchy of Transformations

Projective linear group

Affine group (last row $(0,0,1)$)

Euclidean group (upper left 2×2 orthogonal)

Oriented Euclidean group (upper left 2×2 $\det 1$)

Alternatively, characterize transformation in terms of elements or quantities that are preserved or *invariant*

e.g. Euclidean transformations leave distances unchanged



Class I: Isometries

(iso=same, *metric*=measure)

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} \varepsilon \cos \theta & -\sin \theta & t_x \\ \varepsilon \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \varepsilon = \pm 1$$

orientation preserving: $\varepsilon = 1$

orientation reversing: $\varepsilon = -1$

$$\mathbf{x}' = \mathbf{H}_E \mathbf{x} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0^\top & 1 \end{bmatrix} \mathbf{x} \quad \mathbf{R}^\top \mathbf{R} = \mathbf{I}$$

3DOF (1 rotation, 2 translation)

special cases: pure rotation, pure translation

Invariants: length, angle, area

Class II: Similarities

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} s \cos \theta & -s \sin \theta & t_x \\ s \sin \theta & s \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (\text{isometry} + \text{scale})$$

$$\mathbf{x}' = \mathbf{H}_S \mathbf{x} = \begin{bmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \mathbf{x} \quad \mathbf{R}^\top \mathbf{R} = \mathbf{I}$$

4DOF (1 scale, 1 rotation, 2 translation)

also known as *equi-form* (shape preserving)

metric structure = structure up to similarity (in literature)

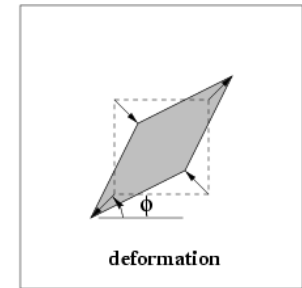
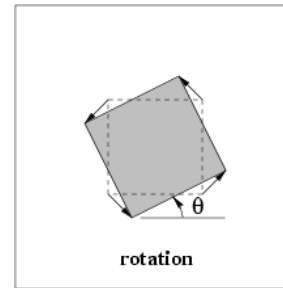
Invariants: ratios of length, angle, ratios of areas,
parallel lines

Class III: Affine Transformations

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\mathbf{x}' = \mathbf{H}_A \mathbf{x} = \begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \mathbf{x}$$

$$\mathbf{A} = \mathbf{R}(\theta)\mathbf{R}(-\phi)\mathbf{D}\mathbf{R}(\phi) \quad \mathbf{D} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$



6DOF (2 scale, 2 rotation, 2 translation)

non-isotropic scaling! (2DOF: scale ratio and orientation)

Invariants: parallel lines, ratios of parallel lengths,
ratios of areas

Class VI: Projective Transformations

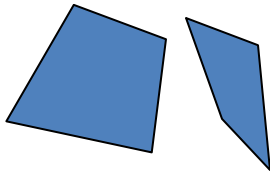
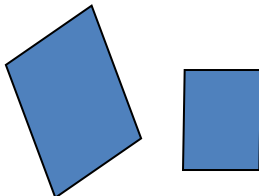
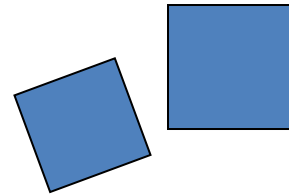
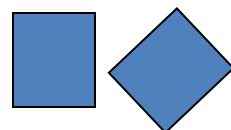
$$\mathbf{x}' = \mathbf{H}_P \mathbf{x} = \begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{v}^\top & v \end{bmatrix} \mathbf{x} \quad \mathbf{v} = (v_1, v_2)^\top$$

8DOF (2 scale, 2 rotation, 2 translation, 2 line at infinity)

Action is non-homogeneous over the plane

Invariants: cross-ratio of four points on a line
(ratio of ratios)

Overview of Transformations

Projective 8dof	$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$		Concurrency, collinearity, order of contact (intersection, tangency, inflection, etc.), cross ratio
Affine 6dof	$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		Parallellism, ratio of areas, ratio of lengths on parallel lines (e.g midpoints), linear combinations of vectors (centroids). The line at infinity l_∞
Similarity 4dof	$\begin{bmatrix} sr_{11} & sr_{12} & t_x \\ sr_{21} & sr_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		Ratios of lengths, angles. The circular points I,J
Euclidean 3dof	$\begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		lengths, areas.

Homework 1

Warp the basketball court from this image to a new image so that it appears as if the new image was taken from directly above



What are we missing?

Image Warping

Slides by Steve Seitz

Image Transformations

image filtering: change **range** of image

$$g(x) = T(f(x))$$

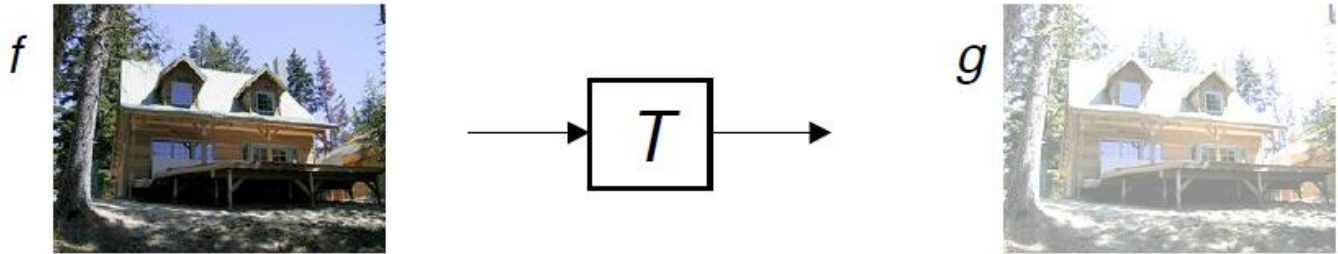
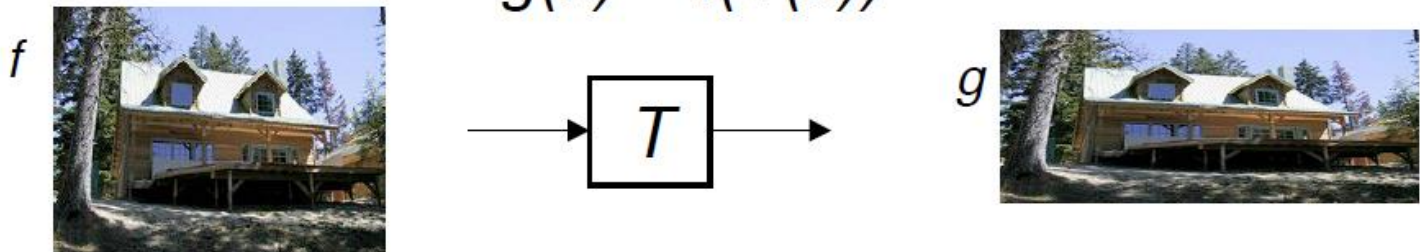


image warping: change **domain** of image

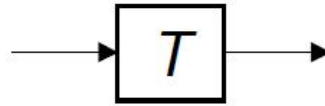
$$g(x) = f(T(x))$$



Parametric (Global) Warping



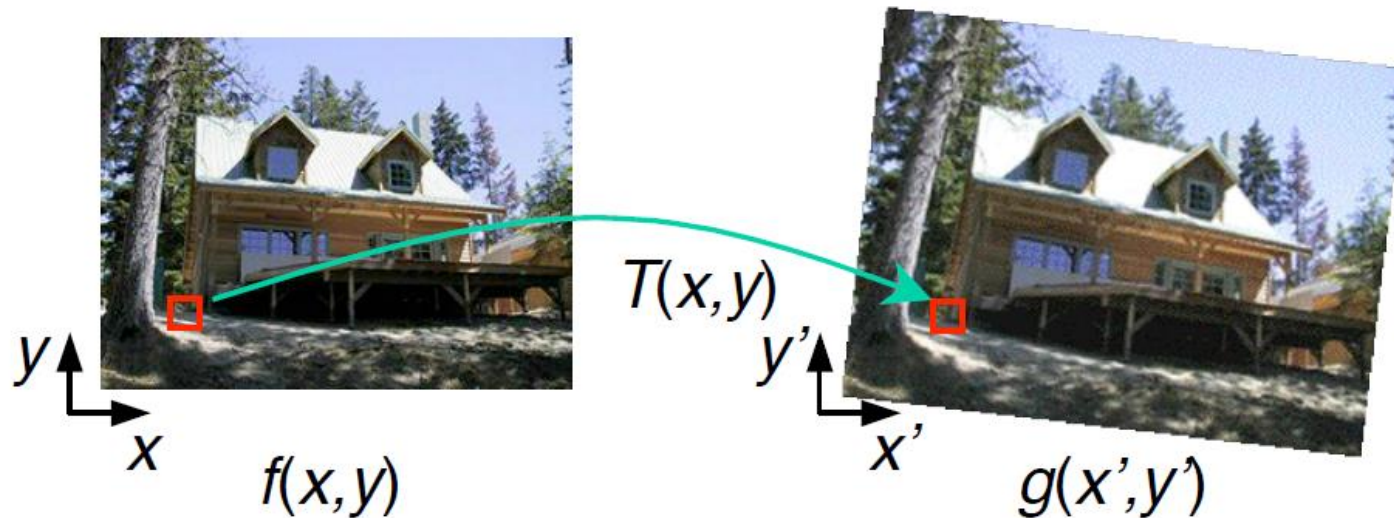
$$\mathbf{p} = (x, y)$$



$$\mathbf{p}' = (x', y')$$

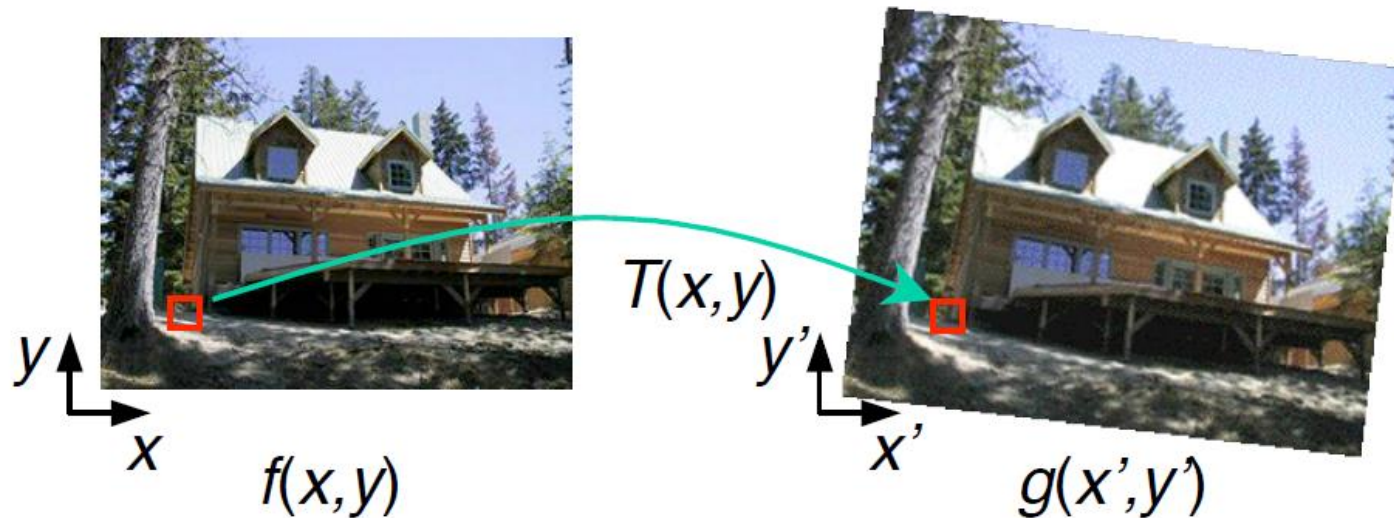
- Transformation T is a coordinate-changing machine:
$$\mathbf{p}' = T(\mathbf{p})$$
- What does it mean that T is global?
 - It is the same for any point $\mathbf{p}$
 - It can be described by just a few numbers (parameters)
- T is represented as a matrix (see prev. slides):
$$\mathbf{p}' = M * \mathbf{p}$$

Image Warping



Given a coordinate transform $(x',y') = h(x,y)$ and a source image $f(x,y)$, how do we compute a transformed image $g(x',y') = f(T(x,y))$?

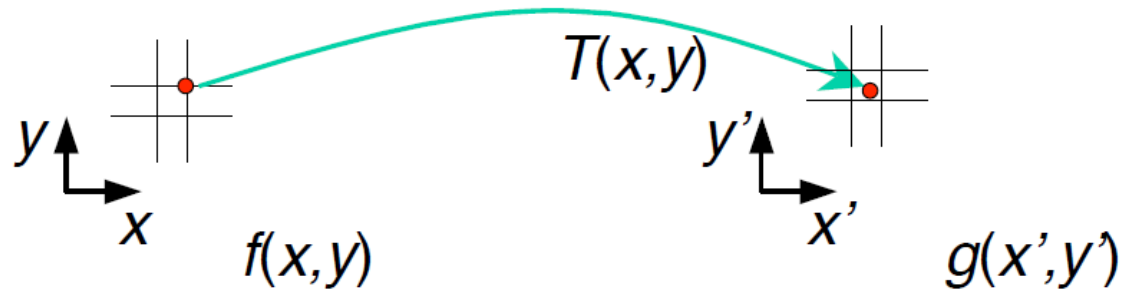
Forward Warping



Send each pixel $f(x, y)$ to its corresponding location $(x', y') = T(x, y)$ in the second image

Q: what if the pixel lands “between” two pixels?

Forward Warping

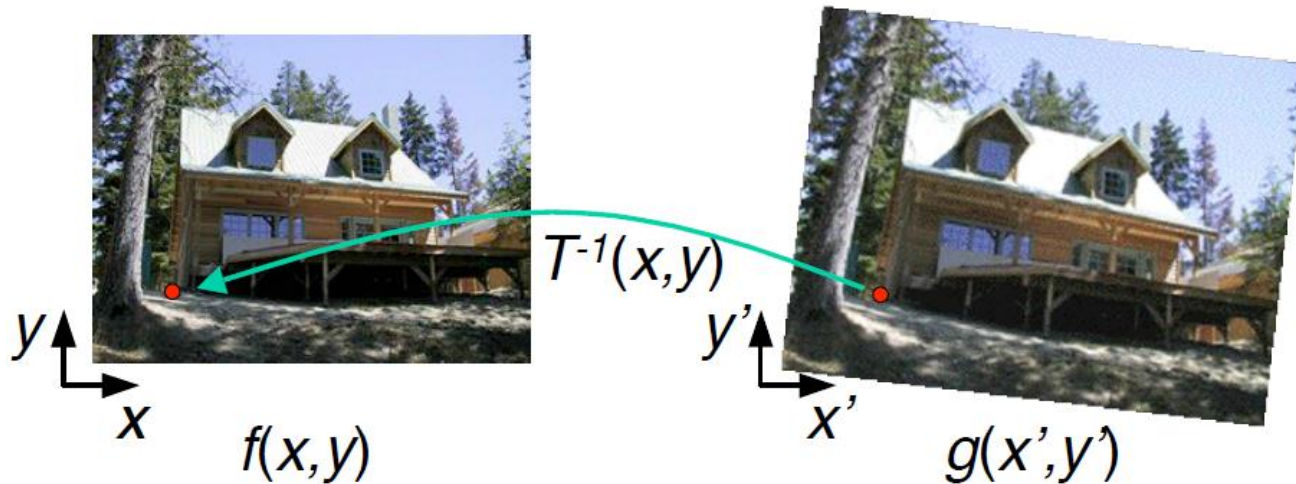


Send each pixel $f(x,y)$ to its corresponding location $(x',y') = T(x,y)$ in the second image

Q: what if the pixel lands “between” two pixels?

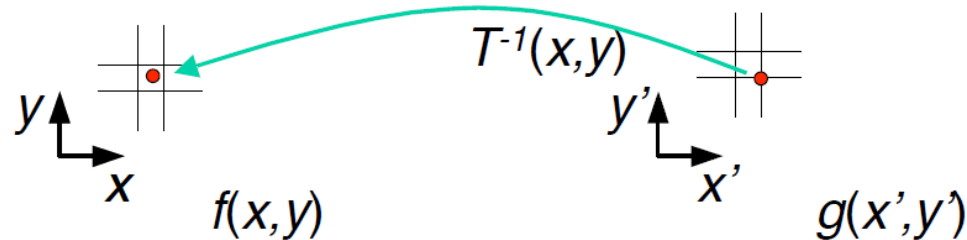
A: Distribute color among neighboring pixels (splatting)

Inverse Warping



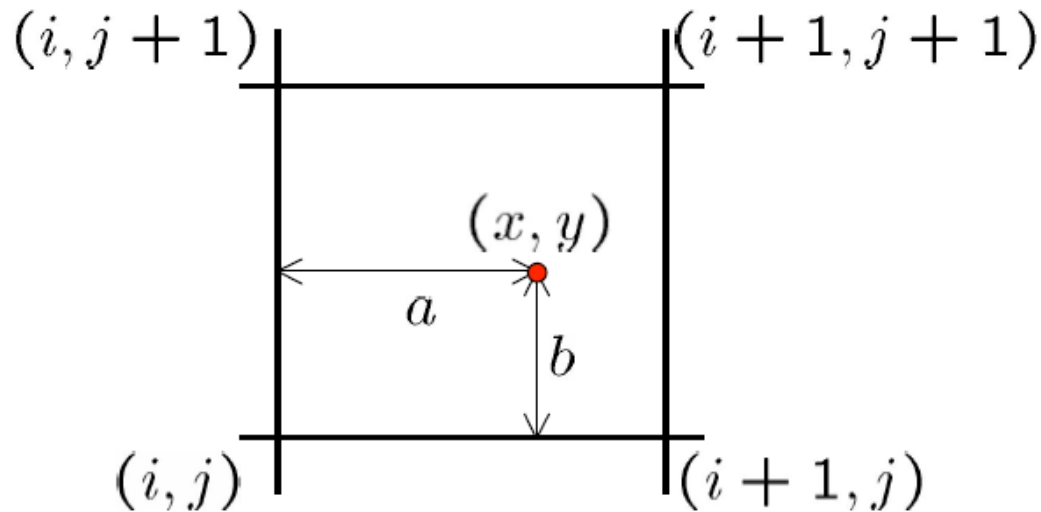
- Get each pixel $g(x',y')$ from its corresponding location $(x,y) = T^{-1}(x',y')$ in the first image
- Q: what if pixel comes from “between” two pixels?

Inverse Warping



- Get each pixel $g(x', y')$ from its corresponding location $(x, y) = T^{-1}(x', y')$ in the first image
- Q: what if pixel comes from “between” two pixels?
- A: interpolate color value from neighbors
 - Bilinear interpolation typically used

Bilinear Interpolation



$$\begin{aligned} f(x, y) = & (1 - a)(1 - b) f[i, j] \\ & + a(1 - b) f[i + 1, j] \\ & + ab f[i + 1, j + 1] \\ & + (1 - a)b f[i, j + 1] \end{aligned}$$

Forward vs. Inverse Warping

- Which is better?
- ...

Parameter Estimation

Slides by R. Hartley, A. Zisserman
and M. Pollefeys

Homography: Number of Measurements Required

- At least as many independent equations as degrees of freedom required
- Example:

$$\mathbf{x}' = \mathbf{H}\mathbf{x}$$

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2 independent equations / point
8 degrees of freedom

$$4 \times 2 \geq 8$$

Approximate solutions

- Minimal solution
 - 4 points yield an exact solution for H
- More points
 - No exact solution, because measurements are inexact (“noise”)
 - Search for “best” according to some cost function
 - Algebraic or geometric/statistical cost

Direct Linear Transformation (DLT)

$$\mathbf{x}'_i = \mathbf{H}\mathbf{x}_i \quad \mathbf{x}'_i \times \mathbf{H}\mathbf{x}_i = 0 \quad \mathbf{x}'_i = (x'_i, y'_i, w'_i)^\top \quad \mathbf{H}\mathbf{x}_i = \begin{pmatrix} \mathbf{h}^{1^\top} \mathbf{x}_i \\ \mathbf{h}^{2^\top} \mathbf{x}_i \\ \mathbf{h}^{3^\top} \mathbf{x}_i \end{pmatrix}$$

$$\mathbf{x}'_i \times \mathbf{H}\mathbf{x}_i = \begin{pmatrix} y'_i \mathbf{h}^{3^\top} \mathbf{x}_i - w'_i \mathbf{h}^{2^\top} \mathbf{x}_i \\ w'_i \mathbf{h}^{1^\top} \mathbf{x}_i - x'_i \mathbf{h}^{3^\top} \mathbf{x}_i \\ x'_i \mathbf{h}^{2^\top} \mathbf{x}_i - y'_i \mathbf{h}^{1^\top} \mathbf{x}_i \end{pmatrix}$$

$$\begin{bmatrix} 0^\top & -w'_i \mathbf{x}_i^\top & y'_i \mathbf{x}_i^\top \\ w'_i \mathbf{x}_i^\top & 0^\top & -x'_i \mathbf{x}_i^\top \\ -y'_i \mathbf{x}_i^\top & x'_i \mathbf{x}_i^\top & 0^\top \end{bmatrix} \begin{pmatrix} \mathbf{h}^1 \\ \mathbf{h}^2 \\ \mathbf{h}^3 \end{pmatrix} = 0$$

$$\mathbf{A}_i \mathbf{h} = 0$$

Direct Linear Transformation (DLT)

Equations are linear in $\mathbf{h}$

$$\mathbf{A}_i \mathbf{h} = 0$$

Only 2 of 3 are linearly independent
(indeed, 2 eq/pt)

$$\begin{bmatrix} 0^\top & -w'_i \mathbf{x}_i^\top & y'_i \mathbf{x}_i^\top \\ w'_i \mathbf{x}_i^\top & 0^\top & -x'_i \mathbf{x}_i^\top \\ -y'_i \mathbf{x}_i^\top & x'_i \mathbf{x}_i^\top & 0^\top \end{bmatrix} \begin{pmatrix} h^1 \\ h^2 \\ h^3 \end{pmatrix} = 0$$

$$x'_i \mathbf{A}_i^1 + y'_i \mathbf{A}_i^2 + w'_i \mathbf{A}_i^3 = 0$$

Direct Linear Transformation (DLT)

$$\begin{bmatrix} 0^\top & -w'_i \mathbf{x}_i^\top & y'_i \mathbf{x}_i^\top \\ w'_i \mathbf{x}_i^\top & 0^\top & -x'_i \mathbf{x}_i^\top \end{bmatrix} \begin{pmatrix} h^1 \\ h^2 \\ h^3 \end{pmatrix} = 0$$

(only drop third row if $w'_i \neq 0$)

- Holds for any homogeneous representation, e.g. $(x'_i, y'_i, 1)$

Direct Linear Transformation (DLT)

- Solving for H $Ah = 0$

$$\begin{bmatrix} A_1 \\ A_2 \\ A_3 \\ A_4 \end{bmatrix} h = 0 \quad \text{Size of } A \text{ is } 8 \times 9, \text{ but rank } 8$$

Trivial solution is $h=0_9^T$ is not interesting
1-D null-space yields solution of interest,
pick for example the one with $\|h\| = 1$

Direct Linear Transformation (DLT)

- Over-determined solution
$$\begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_n \end{bmatrix} h = 0$$

No exact solution because of inexact measurement
i.e. “noise”

Find approximate solution

- Additional constraint needed to avoid 0, e.g. $\|h\| = 1$
- $Ah = 0$ not possible, so minimize $\|Ah\|$

DLT Algorithm

Objective

Given $n \geq 4$ 2D to 2D point correspondences $\{x_i \leftrightarrow x_i'\}$, determine the 2D homography matrix H such that $x_i' = Hx_i$

Algorithm

- (i) For each correspondence $x_i \leftrightarrow x_i'$ compute A_i . Usually only two first rows needed.
- (ii) Assemble n 2×9 matrices A_i into a single $2n \times 9$ matrix A
- (iii) Obtain SVD of A . Solution for h is last column of V
- (iv) Determine H from h

Inhomogeneous solution

Since h can only be computed up to scale,
pick $h_j=1$, e.g. $h_9=1$, and solve for 8-vector $\tilde{h}$

$$\begin{bmatrix} 0 & 0 & 0 & -x_i w_i' & -y_i w_i' & -w_i w_i' & x_i y_i' & y_i y_i' \\ x_i w_i' & y_i w_i' & w_i w_i' & 0 & 0 & 0 & -x_i x_i' & -y_i x_i' \end{bmatrix} \tilde{h} = \begin{pmatrix} -w_i y_i' \\ w_i x_i' \end{pmatrix}$$

Solve using Gaussian elimination (4 points) or
using linear least-squares (more than 4 points)

However, if $h_9=0$ this approach fails

Also poor results if h_9 close to zero

Therefore, not recommended

Normalizing Transformations

- Since DLT is not invariant to transformations, what is a good choice of coordinates?

e.g.

- Translate centroid to origin
- Scale to a $\sqrt{2}$ average distance to the origin
- Independently on both images

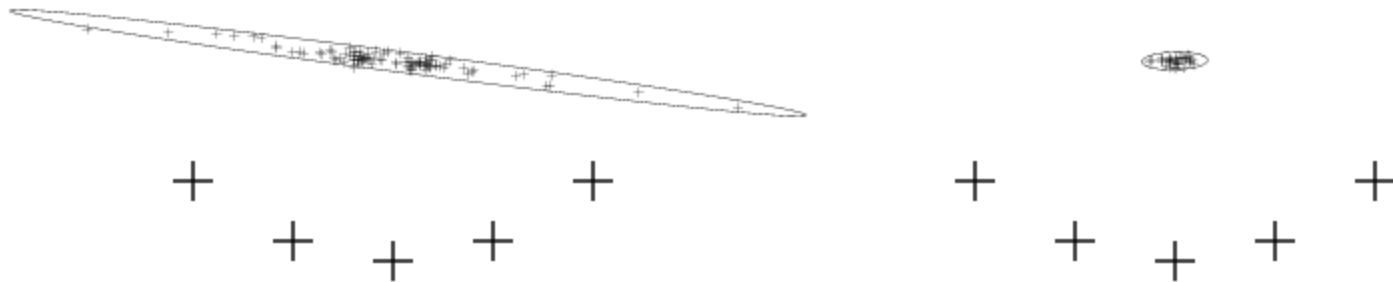
$$T_{\text{norm}} = \begin{bmatrix} w+h & 0 & w/2 \\ 0 & w+h & h/2 \\ 0 & 0 & 1 \end{bmatrix}^{-1}$$

Importance of Normalization

$$\begin{bmatrix} 0 & 0 & 0 & -x'_i & -y'_i & -1 & y'_i x_i & y'_i y_i & y'_i \\ x_i & y_i & 1 & 0 & 0 & 0 & -x'_i x_i & -x'_i y_i & -x'_i \end{bmatrix} \begin{pmatrix} h^1 \\ h^2 \\ h^3 \end{pmatrix} = 0$$

$\sim 10^2 \quad \sim 10^2 \quad 1 \quad \sim 10^2 \quad \sim 10^2 \quad 1 \quad \sim 10^4 \quad \sim 10^4 \quad \sim 10^2$

orders of magnitude difference!



Monte Carlo simulation

for identity computation based on 5 points

(not normalized ↔ normalized)

Normalized DLT Algorithm

Objective

Given $n \geq 4$ 2D to 2D point correspondences $\{x_i \leftrightarrow x'_i\}$, determine the 2D homography matrix H such that $x'_i = Hx_i$

Algorithm

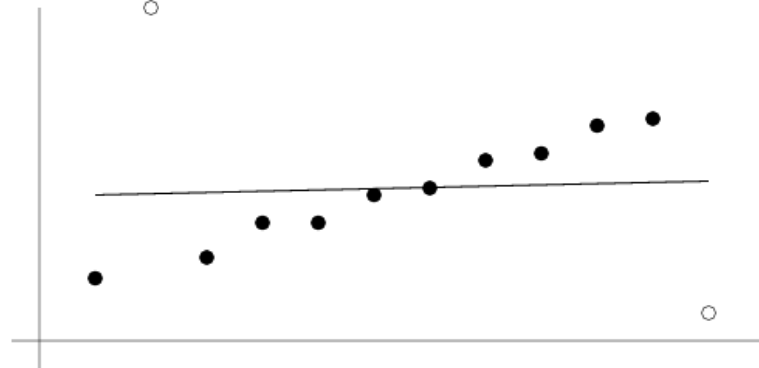
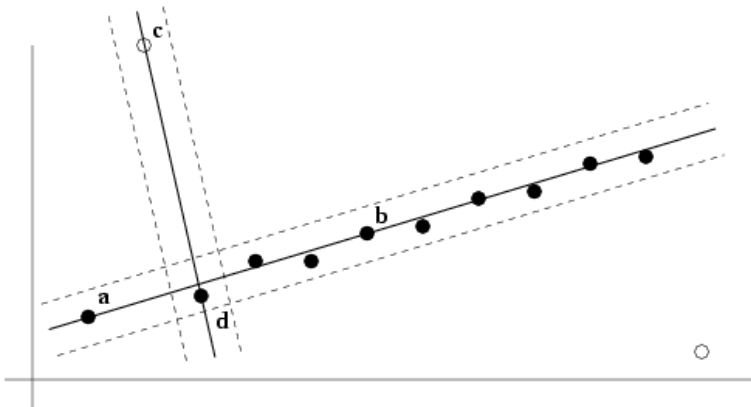
- (i) Normalize points $\tilde{x}_i = T_{\text{norm}} x_i, \tilde{x}'_i = T'_{\text{norm}} x'_i$
- (ii) Apply DLT algorithm to $\tilde{x}_i \leftrightarrow \tilde{x}'_i$,
- (iii) Denormalize solution $H = T_{\text{norm}}'^{-1} \tilde{H} T_{\text{norm}}$

RANSAC

Slides by R. Hartley, A. Zisserman
and M. Pollefeys

Robust Estimation

- What if set of matches contains gross outliers?



RANSAC

Objective

Robust fit of model to data set S which contains outliers

Algorithm

- (i) Randomly select a sample of s data points from S and instantiate the model from this subset.
- (ii) Determine the set of data points S_i which are within a **distance threshold** t of the model. The set S_i is the consensus set of samples and defines the inliers of S .
- (iii) If the subset of S_i is greater than some threshold T , re-estimate the model using all the points in S_i and terminate
- (iv) If the size of S_i is less than T , select a new subset and repeat the above.
- (v) After N trials the largest consensus set S_i is selected, and the model is re-estimated using all the points in the subset S_i

How Many Samples?

Choose N so that, with probability p , at least one random sample is free from outliers. e.g. $p=0.99$

$$\left(1 - (1 - e)^s\right)^N = 1 - p$$

$$N = \log(1 - p) / \log\left(1 - (1 - e)^s\right)$$

proportion of outliers e							
s	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

Acceptable Consensus Set

- Typically, terminate when inlier ratio reaches expected ratio of inliers

$$T = (1 - e)n$$

Adaptively Determining the Number of Samples

e is often unknown a priori, so pick worst case, e.g. 50%, and adapt if more inliers are found, e.g. 80% would yield $e=0.2$

- $N=\infty$, $sample_count=0$
- While $N > sample_count$ repeat
 - Choose a sample and count the number of inliers
 - Set $e=1-(\text{number of inliers})/(\text{total number of points})$
 - Recompute N from e
 - Increment the $sample_count$ by 1
- Terminate

$$\left(N = \log(1 - p) / \log(1 - (1 - e)^s) \right)$$

Other robust algorithms

- RANSAC maximizes number of inliers
- LMedS minimizes median error
- Not recommended: case deletion, iterative least-squares, etc.

Automatic Computation of H

Objective

Compute homography between two images

Algorithm

- (i) **Interest points:** Compute interest points in each image
- (ii) **Putative correspondences:** Compute a set of interest point matches based on some similarity measure
- (iii) **RANSAC robust estimation:** Repeat for N samples
 - (a) Select 4 correspondences and compute H
 - (b) Calculate the distance $d_{\perp}$ for each putative match
 - (c) Compute the number of inliers consistent with H ($d_{\perp} < t$)Choose H with most inliers
- (iv) **Optimal estimation:** re-estimate H from all inliers by minimizing ML cost function with Levenberg-Marquardt
- (v) **Guided matching:** Determine more matches using prediction by computed H

Optionally iterate last two steps until convergence

Determine Putative Correspondences

- Compare interest points
Similarity measure:
 - SAD, SSD, ZNCC in small neighborhood
- If motion is limited, only consider interest points with similar coordinates

Example: robust computation

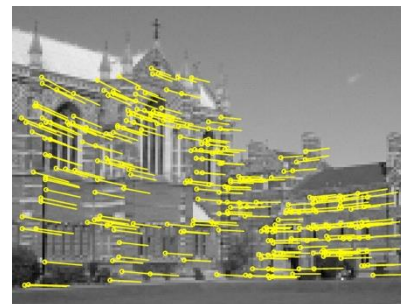
#in	1-e	adapt. N
6	2%	20M
10	3%	2.5M
44	16%	6,922
58	21%	2,291
73	26%	911
151	56%	43



Interest points
(500/image)
(640x480)



Putative correspondences (268)
(Best match, SSD < 20, ± 320)
Outliers (117)
($t=1.25$ pixel; 43 iterations)



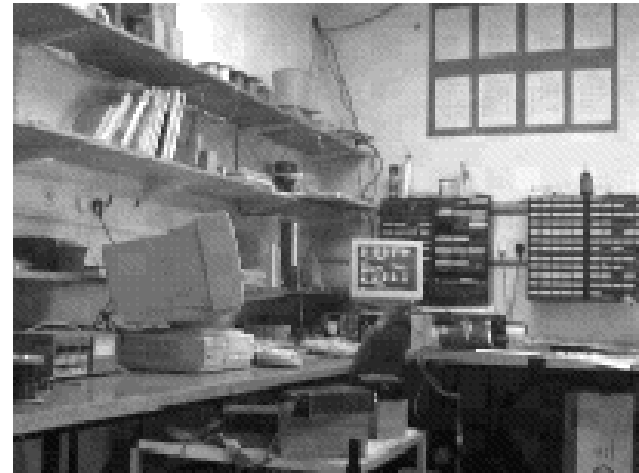
Inliers (151)

Final inliers (262)

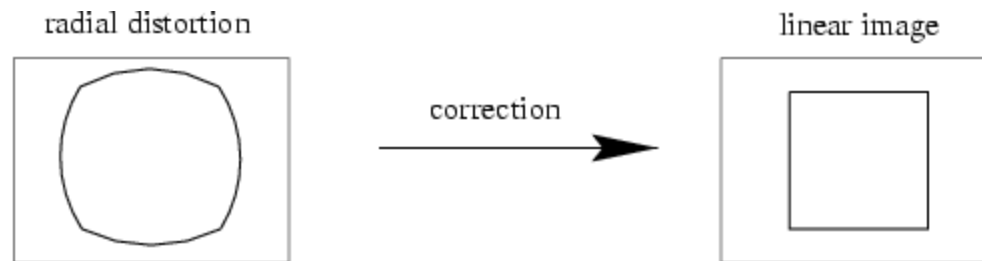
Radial Distortion and Undistortion

Slides by R. Hartley, A. Zisserman
and M. Pollefeys

Radial Distortion



short and long focal length







Typical Undistortion Model

Correction of distortion

$$\hat{x} = x_c + L(r)(x - x_c) \quad \hat{y} = y_c + L(r)(y - y_c)$$

Choice of the distortion function and center

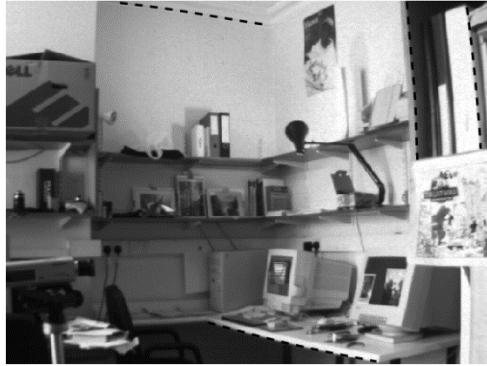
$$\begin{aligned} x &= x_o + (x_o - c_x)(K_1 r^2 + K_2 r^4 + \dots) \\ y &= y_o + (y_o - c_y)(K_1 r^2 + K_2 r^4 + \dots) \end{aligned}$$

$$r = (x_o - c_x)^2 + (y_o - c_y)^2 \quad .$$

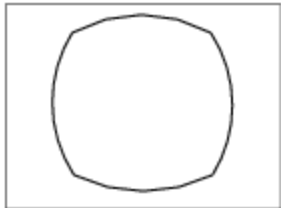
Computing the parameters of the distortion function

- (i) Minimize with additional unknowns
- (ii) Straighten lines
- (iii) ...

Why Undistort?



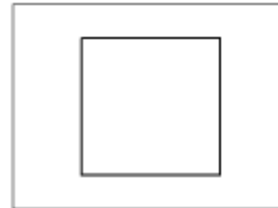
radial distortion



correction



linear image



$$(\tilde{x}, \tilde{y}, 1)^\top = [\mathbf{I} \mid \mathbf{0}] \mathbf{X}_{\text{cam}}$$

$$\begin{pmatrix} x_d \\ y_d \end{pmatrix} = L(\tilde{r}) \begin{pmatrix} \tilde{x} \\ \tilde{y} \end{pmatrix}$$

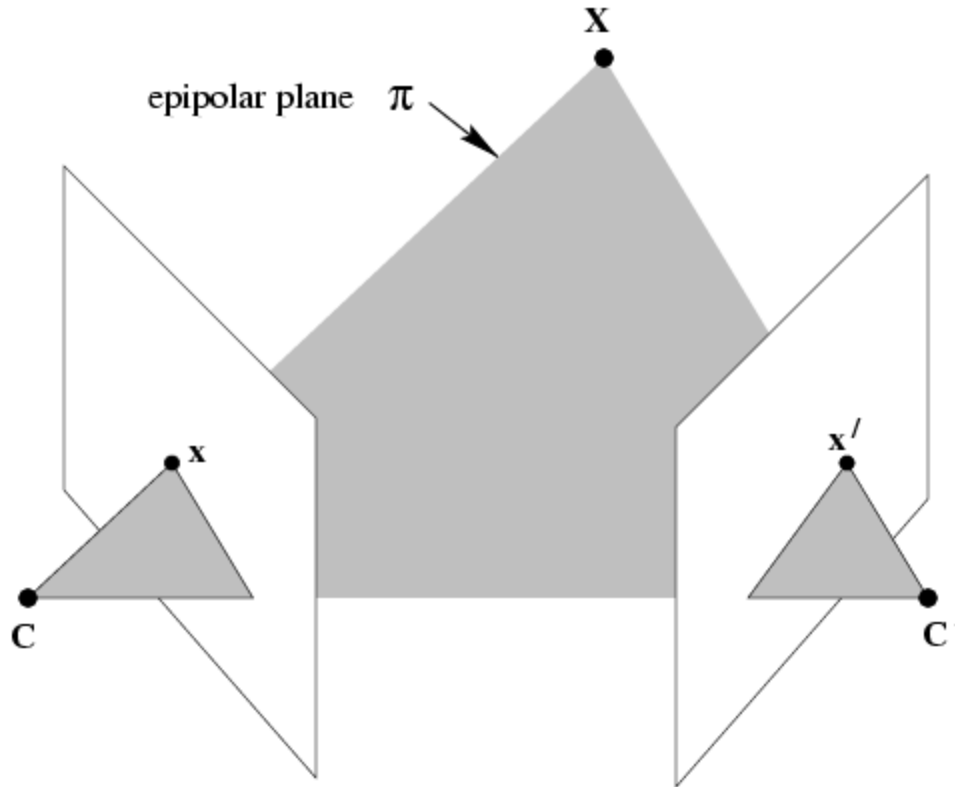
Two-View Geometry

Slides by R. Hartley, A. Zisserman and M. Pollefeys

Three questions:

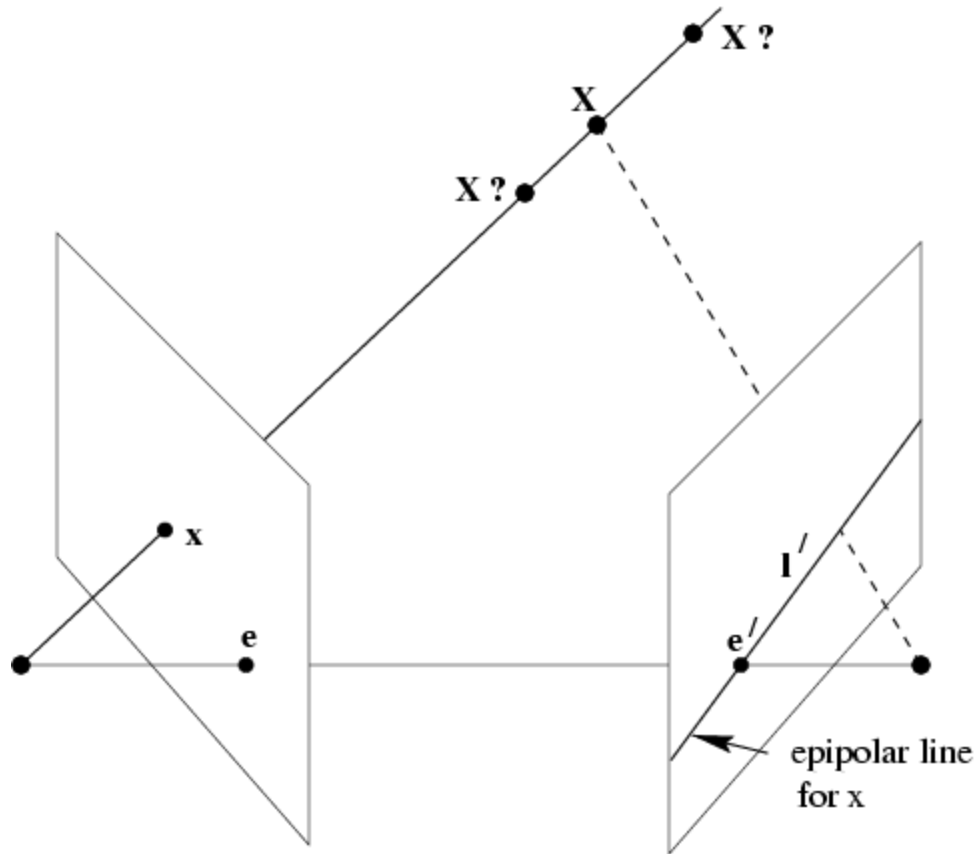
- (i) **Correspondence geometry:** Given an image point X in the first image, how does this constrain the position of the corresponding point X' in the second image?
- (ii) **Camera geometry (motion):** Given a set of corresponding image points $\{x_i \leftrightarrow x'_i\}$, $i=1, \dots, n$, what are the cameras P and P' for the two views?
- (iii) **Scene geometry (structure):** Given corresponding image points $x_i \leftrightarrow x'_i$ and cameras P, P' , what is the position of (their pre-image) X in space?

The Epipolar Geometry



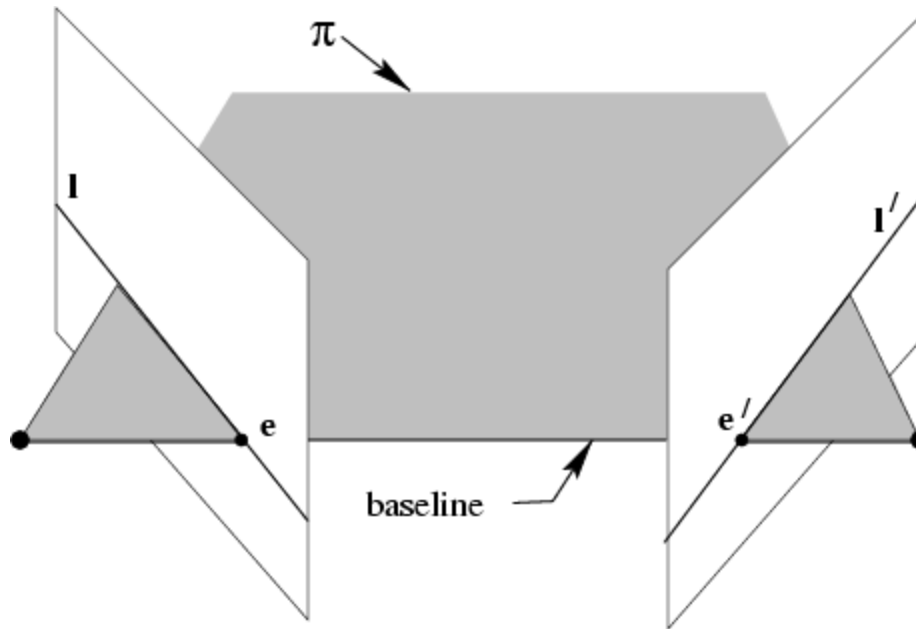
C, C', x, x' and X are coplanar

The Epipolar Geometry



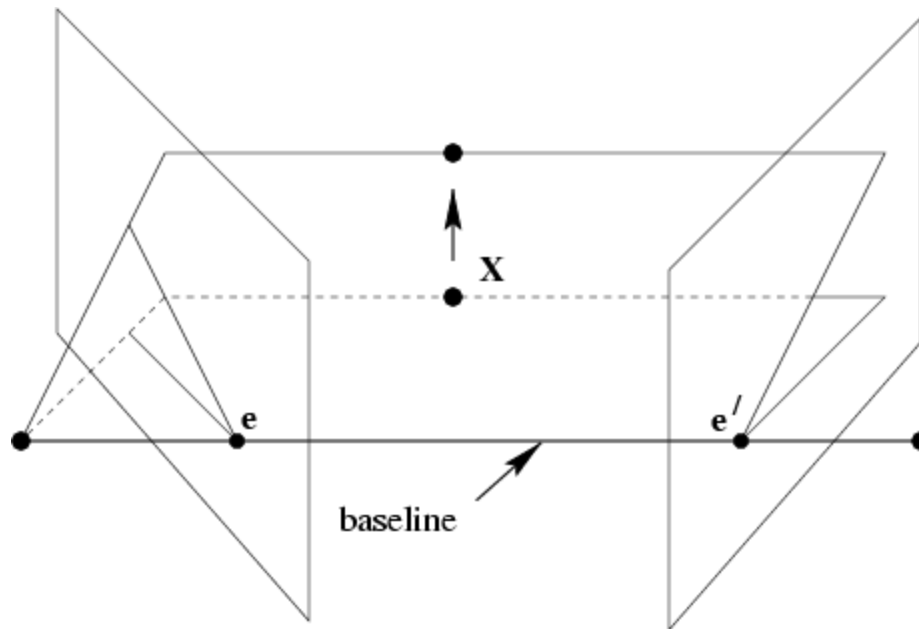
What if only C, C', x are known?

The Epipolar Geometry



All points on π project on l and l'

The Epipolar Geometry



Family of planes π and lines l and l'
Intersection in e and e'

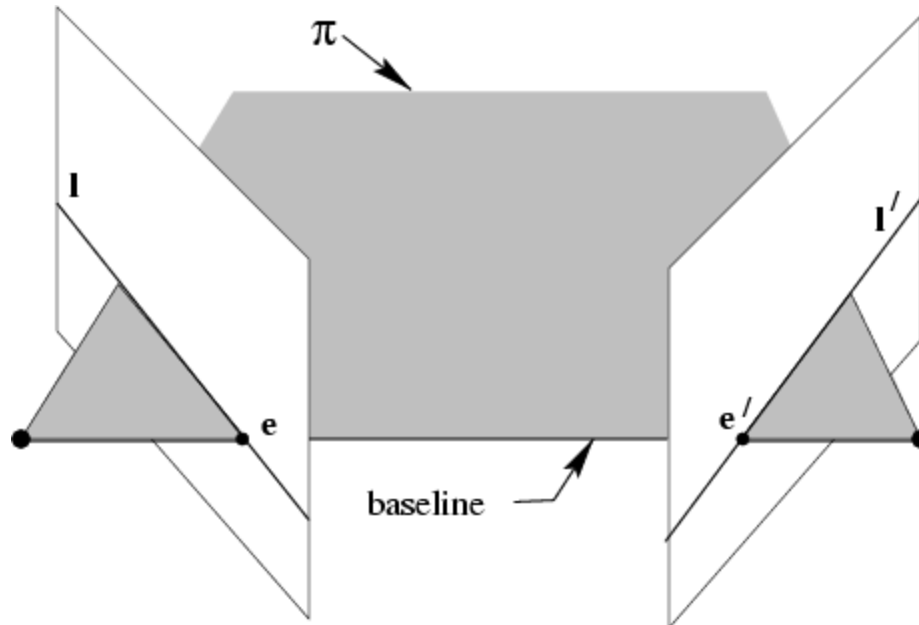
The Epipolar Geometry

epipoles e, e'

= intersection of baseline with image plane

= projection of projection center in other image

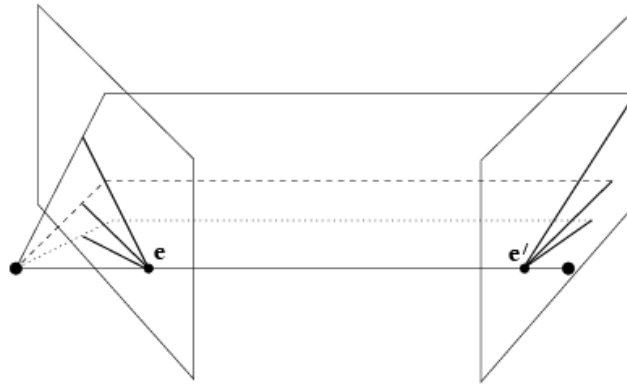
= vanishing point of camera motion direction



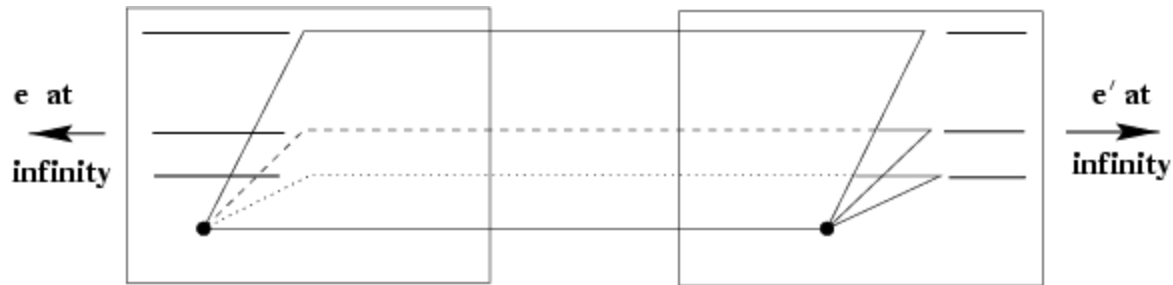
an epipolar plane = plane containing baseline (1-D family)

an epipolar line = intersection of epipolar plane with image
(always come in corresponding pairs)

Example: Converging Cameras

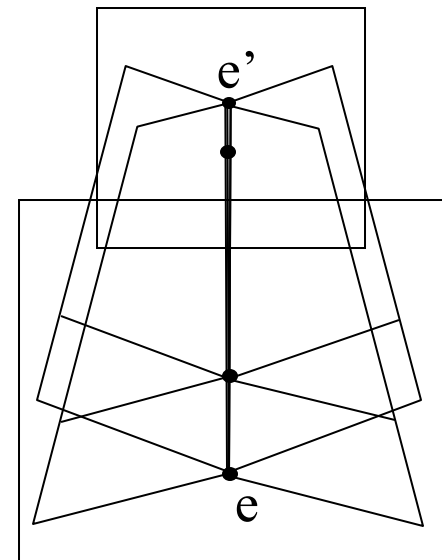
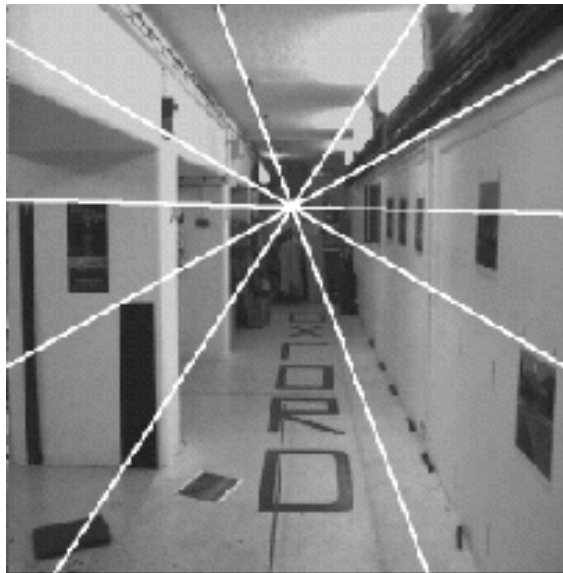
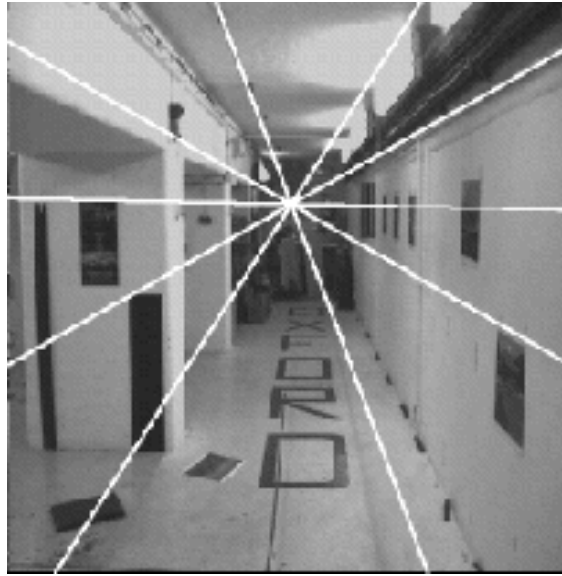


Example: Motion Parallel to Image Plane



(simple for stereo → rectification)

Example: Forward Motion



The Fundamental Matrix F

algebraic representation of epipolar geometry

$$\mathbf{x} \mapsto \mathbf{l}'$$

we will see that mapping is a (singular) correlation
(i.e. projective mapping from points to lines)
represented by the fundamental matrix F

The Fundamental Matrix F

correspondence condition

The fundamental matrix satisfies the condition that for any pair of corresponding points $x \leftrightarrow x'$ in the two images

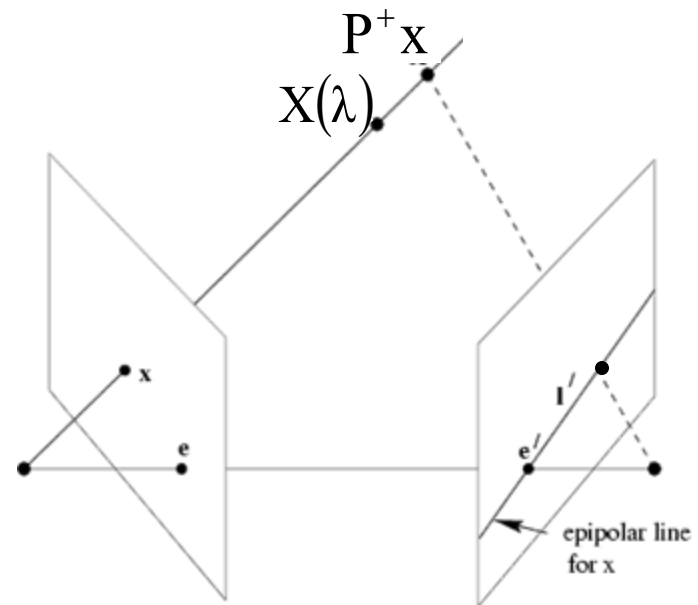
$$x'^T F x = 0 \quad (x'^T l' = 0)$$

The Fundamental Matrix F

$$X(\lambda) = P^+ x + \lambda C \quad (PP^+ = I)$$

$$l = P' C \times P' P^+ x$$

$$F = [e']_{\times} P' P^+$$



(note: doesn't work for $C=C' \Rightarrow F=0$)

The Fundamental Matrix F

F is the unique 3×3 rank 2 matrix that satisfies $x'^T F x = 0$ for all $x \leftrightarrow x'$

- (i) **Transpose:** if F is fundamental matrix for (P, P') , then F^T is fundamental matrix for (P', P)
- (ii) **Epipolar lines:** $l' = Fx$ & $l = F^T x'$
- (iii) **Epipoles:** on all epipolar lines, thus $e'^T F x = 0, \forall x \Rightarrow e'^T F = 0$, similarly $F e = 0$
- (iv) F has 7 d.o.f. , i.e. $3 \times 3 - 1(\text{homogeneous}) - 1(\text{rank } 2)$
- (v) F is a correlation, projective mapping from a point x to a line $l' = Fx$ (not a proper correlation, i.e. not invertible)

Two View Geometry Computation: Linear Algorithm

For every match (m,m'): $x'^T F X = 0$

$$x' x f_{11} + x' y f_{12} + x' f_{13} + y' x f_{21} + y' y f_{22} + y' f_{23} + x f_{31} + y f_{32} + f_{33} = 0$$

separate known from unknown

$$\underbrace{[x' x, x' y, x', y' x, y' y, y', x, y, 1]}_{\text{(data)}} \underbrace{[f_{11}, f_{12}, f_{13}, f_{21}, f_{22}, f_{23}, f_{31}, f_{32}, f_{33}]^T}_{\substack{\text{(unknowns)} \\ \text{(linear)}}} = 0$$

$$\begin{bmatrix} x'_1 x_1 & x'_1 y_1 & x'_1 & y'_1 x_1 & y'_1 y_1 & y'_1 & x_1 & y_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x'_n x_n & x'_n y_n & x'_n & y'_n x_n & y'_n y_n & y'_n & x_n & y_n & 1 \end{bmatrix} \mathbf{f} = 0$$

$$A \mathbf{f} = 0$$

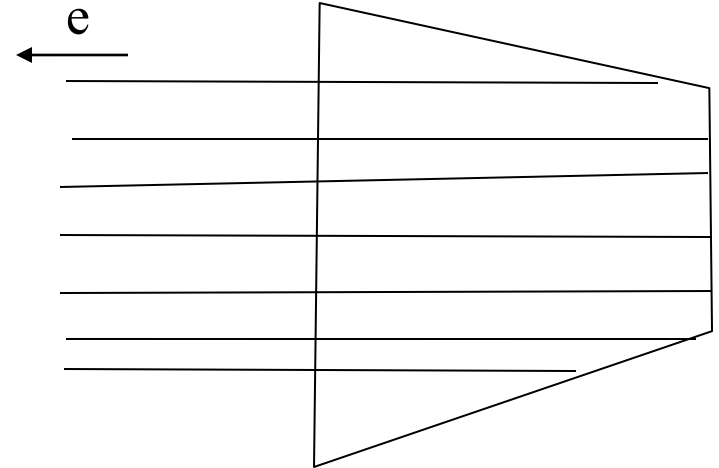
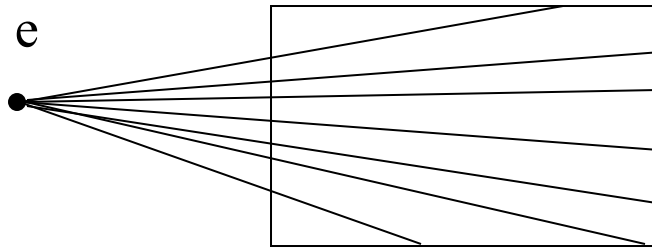
Benefits from having F

- Given a pixel in one image, the corresponding pixel has to lie on epipolar line
- Search space reduced from 2-D to 1-D

Image Pair Rectification

simplify stereo matching
by warping the images

Apply projective transformation so that epipolar lines
correspond to horizontal scanlines



$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = H e$$

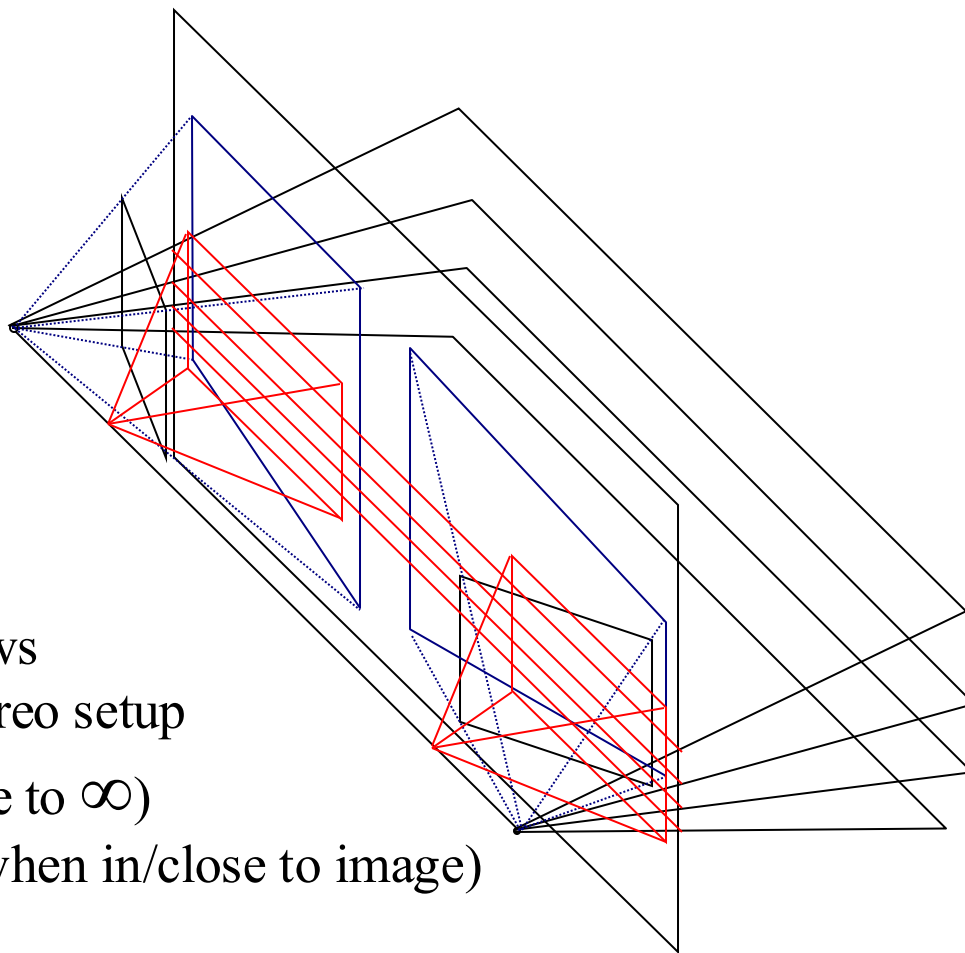
map epipole e to $(1,0,0)$

try to minimize image distortion

problem when epipole in (or close to) the image

Planar Rectification

(standard approach)



Bring two views
to standard stereo setup

(moves epipole to ∞)

(not possible when in/close to image)

